



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

Trabajo fin de Grado

Grado en Ingeniería Informática (Ingeniería del Software)

BargAIIn: Sistema inteligente de optimización de la cesta de la compra

**Realizado por
Nicolás Parrilla Geniz**

**Dirigido por
Juan Vicente Gutiérrez Santacreu**

**Departamento
Matemática Aplicada I**

Sevilla, Junio 2026

Resumen

BargAIn es una plataforma de compra inteligente orientada a optimizar la cesta de la compra del consumidor combinando tres variables de decisión: precio, distancia y tiempo. El proyecto desarrolla una solución full-stack con backend en Django/DRF, base de datos PostgreSQL + PostGIS, cliente móvil en React Native (Expo), y companion web para el portal de comercios.

La propuesta integra cuatro capacidades diferenciales en una única aplicación: (1) comparación multi-tienda de precios, (2) optimización multicriterio de ruta con OR-Tools, (3) digitalización de listas y tickets mediante OCR con Google Cloud Vision, y (4) asistente conversacional basado en LLM (Google Gemini) vía backend proxy.

Desde la perspectiva de ingeniería del software, el trabajo cubre el ciclo completo: análisis de requisitos, diseño arquitectónico, implementación modular, pruebas unitarias/integración/E2E, despliegue en staging y documentación técnica. La arquitectura se organiza en dominios desacoplados (users, products, stores, prices, shopping_lists, optimizer, ocr, assistant, business, notifications), con tareas asíncronas en Celery y Redis para procesos pesados.

Como resultado, se obtiene un prototipo funcional validado en entorno de desarrollo y staging, con cobertura de pruebas en módulos críticos, flujos E2E automatizados para el web companion, y verificaciones manuales en flujos móviles dependientes de GPS/cámara. La memoria documenta además limitaciones, decisiones arquitectónicas y líneas de evolución para convertir BargAIn en un producto escalable.

Abstract

BargAI is an intelligent shopping platform focused on optimizing grocery baskets by combining three decision variables: price, distance, and time. The project delivers a full-stack solution with a Django/DRF backend, PostgreSQL + PostGIS geospatial storage, React Native (Expo) mobile client, and a web companion for small businesses.

The proposal integrates four key capabilities in a single system: (1) multi-store price comparison, (2) multicriteria route optimization with OR-Tools, (3) OCR-based list/ticket digitization using Google Cloud Vision, and (4) a conversational assistant powered by an LLM (Google Gemini) through a backend proxy.

From a software engineering standpoint, the work covers the whole lifecycle: requirements analysis, architecture design, modular implementation, unit/integration/E2E testing, staging deployment, and technical documentation. The architecture is organized into decoupled domains, with asynchronous processing delegated to Celery workers over Redis.

The final outcome is a functional prototype validated through automated and manual tests, with clear traceability of technical decisions, current limitations, and future evolution paths toward production readiness.

Agradecimientos

A mi tutor, Juan Vicente Gutiérrez Santacreu, por su orientación constante, su criterio técnico y su apoyo durante todo el desarrollo del TFG.

A mi familia y personas cercanas, por su paciencia y motivación en las fases de mayor carga de trabajo.

A la comunidad de software libre y a la documentación técnica de las herramientas utilizadas, que han sido clave para avanzar con rigor en el diseño, la implementación y la validación de BargAIn.

Índice general

Índice general	IV
Índice de cuadros	VII
Índice de figuras	VIII
1 Introducción y objetivos del proyecto	1
1.1 Contexto y motivación	1
1.2 Problema abordado	1
1.3 Objetivo principal	1
1.4 Objetivos específicos	1
1.5 Alcance del proyecto	2
1.6 Tecnologías clave	2
1.7 Estructura de la memoria	3
2 Análisis de antecedentes y aportación realizada	4
2.1 Contexto del sector	4
2.2 Estado del arte en comparación de precios	4
2.3 Tecnologías relevantes para BargAIIn	4
2.3.1 Geoespacial y rutas	4
2.3.2 Ingesta de datos de precio	4
2.3.3 OCR y asistente	5
2.4 Aportación diferencial de BargAIIn	5
3 Comparación con otras alternativas	6
3.1 Competidores considerados	6
3.2 Matriz funcional resumida	6
3.3 Brechas estratégicas detectadas	7
3.4 Posicionamiento de BargAIIn	7
3.5 Riesgos competitivos	7
4 Análisis temporal y costes de desarrollo	8
4.1 Planificación temporal	8
4.2 Metodología de trabajo	8
4.3 Estimación de costes	9
4.4 Riesgos y mitigaciones	9
4.5 Resultado de planificación	9

5	Análisis de requisitos	10
5.1	Actores del sistema	10
5.2	Requisitos de información	10
5.3	Requisitos funcionales	10
5.4	Criterios de aceptación del producto	11
5.5	Requisitos no funcionales	11
5.6	Reglas de negocio	12
5.7	Diagramas de casos de uso	12
5.8	Trazabilidad	12
6	Diseño e Implementación	15
6.1	Arquitectura del Sistema	15
6.1.1	Visión general	15
6.1.2	Patrón de organización interna del backend	15
6.1.3	Comunicación entre componentes	16
6.2	Modelo de Datos	16
6.2.1	Módulo de usuarios	16
6.2.2	Módulo de productos	17
6.2.3	Módulo de tiendas	17
6.2.4	Módulo de precios	17
6.2.5	Módulo del optimizador	18
6.2.6	Índices clave	18
6.2.7	Modelo entidad-relación	18
6.2.8	Diagrama de clases	18
6.3	Diseño de la API REST	18
6.3.1	Estructura general	18
6.3.2	Endpoints principales	20
6.3.3	Autenticación y autorización	20
6.4	Implementación del Backend	20
6.4.1	Módulo de notificaciones	20
6.4.2	Módulo de portal business	21
6.5	El Algoritmo de Optimización de Rutas	21
6.5.1	Formulación del problema	21
6.5.2	Función de scoring multicriterio	21
6.5.3	Pipeline de ejecución	21
6.5.4	Diagrama de secuencia del optimizador	22
6.5.5	Integración con OR-Tools	22
6.5.6	Complejidad y rendimiento	22
6.6	Módulo de Web Scraping	24
6.7	Módulo OCR	24
6.7.1	Diagrama de secuencia OCR	24
6.8	Integración del Asistente LLM	24
6.9	Infraestructura y Despliegue	26
6.9.1	Contenedores Docker	26
6.9.2	Pipeline CI/CD	26
6.9.3	Variables de entorno	26
6.10	Diseño de la Interfaz de Usuario	26
6.10.1	Sistema de diseño	26
6.10.2	Pantallas principales	27
6.10.3	Adaptación a uso web (responsive y conveniencias de escritorio)	27

7	Manual de Usuario	29
7.1	Introducción	29
7.2	Requisitos de uso	29
7.3	Flujo de acceso	29
7.3.1	Pantallas de acceso	30
7.4	Pantalla principal (Home)	30
7.5	Gestión de listas de compra	30
7.6	Catálogo y detalle de producto	31
7.7	Mapa de tiendas	31
7.8	Alertas y notificaciones	32
7.9	Perfil y preferencias	32
7.10	Ruta optimizada persistente y recálculo	32
7.11	Módulo OCR: captura de ticket	32
7.12	Asistente LLM	34
7.13	Checklist en bloqueo de iPhone	34
7.14	Portal Business (web)	34
7.15	Uso desde el navegador web	35
7.16	Galería de capturas de la aplicación	35
7.16.1	Aplicación móvil (usuario)	35
7.16.2	Aplicación web (usuario)	40
7.16.3	Portal web para PYMEs	43
7.17	Resolución de incidencias comunes	47
8	Pruebas y Resultados	48
8.1	Objetivo	48
8.2	Estrategia de testing	48
8.3	Entorno de ejecución	48
8.4	Suite de tests backend	49
8.4.1	Tests de integración	49
8.4.2	Cobertura	49
8.5	Tests E2E con Playwright	49
8.5.1	Configuración Playwright	50
8.6	UAT manual en dispositivo móvil	50
8.7	Validación de Requisitos No Funcionales	50
8.7.1	RNF-002: Disponibilidad $\geq 99\%$ en staging	50
8.7.2	RNF-004: Usabilidad WCAG 2.1 AA y máximo 3 taps	51
8.7.3	RNF-005: Escalabilidad para 10.000 usuarios	51
8.8	Resumen de resultados	51
8.9	Conclusión	52
9	Conclusiones	53
9.1	Grado de cumplimiento	53
9.2	Aportaciones principales del trabajo	53
9.3	Limitaciones	54
9.4	Lecciones aprendidas	54
9.5	Trabajo futuro	55
9.6	Cierre	55
	Referencias	56

Índice de cuadros

3.1	Comparativa de capacidades clave	6
4.1	Resumen por fases del proyecto	8
4.2	Coste estimado del desarrollo	9
5.1	Agrupación de requisitos funcionales	11
6.1	Índices principales de la base de datos	18
6.2	Endpoints de autenticación y usuarios	20
6.3	Endpoints del optimizador, OCR y asistente	20
6.4	Tiempos estimados de ejecución del optimizador	22
6.5	Paleta de colores del sistema de diseño	27
8.1	Suite de tests de integración backend	49
8.2	Suite de tests E2E con Playwright	50
8.3	Resultados UAT en dispositivo móvil	50
8.4	Resumen de resultados de la estrategia de testing	51

Índice de figuras

5.1	Casos de uso del Consumidor	13
5.2	Casos de uso del Comercio y Administrador	14
6.1	Arquitectura por capas de BargAIn	16
6.2	Diagrama entidad-relación de BargAIn	19
6.3	Diagrama de clases del dominio	19
6.4	Diagrama de secuencia del proceso de optimización de ruta	23
6.5	Diagrama de secuencia del flujo OCR	25
7.1	Pantallas de inicio de sesión y registro	30
7.2	Pantalla principal — Dashboard	30
7.3	Pantalla de gestión de lista de compra	31
7.4	Pantalla del optimizador con comparativa de rutas	33
7.5	Flujo OCR: captura de ticket (izquierda) y revisión de ítems (derecha) . . .	33
7.6	Pantalla del asistente conversacional LLM	34
7.7	Móvil — Inicio de sesión (izquierda) y registro (derecha)	35
7.8	Móvil — Pantalla principal (izquierda) y catálogo de productos (derecha) .	36
7.9	Móvil — Comparativa de precios por tienda (izquierda) y propuesta de producto (derecha)	36
7.10	Móvil — Listas de la compra (izquierda) y detalle de una lista (derecha) . .	36
7.11	Móvil — Plantillas de lista (izquierda) y ruta optimizada (derecha)	37
7.12	Móvil — Módulo OCR: captura de ticket (izquierda) y revisión de ítems (derecha)	37
7.13	Móvil — Mapa de tiendas (izquierda) y perfil de tienda (derecha)	37
7.14	Móvil — Tiendas favoritas (izquierda) y asistente conversacional (derecha) .	38
7.15	Móvil — Bandeja de notificaciones (izquierda) y alertas de precio (derecha)	39
7.16	Móvil — Perfil de usuario (izquierda) y configuración del optimizador (de- recha)	39
7.17	Móvil — Edición de perfil (izquierda) y cambio de contraseña (derecha) . .	39
7.18	Web — Pantalla principal con accesos horizontales (izquierda) y catálogo en rejilla multicolumna con <code>Cmd/Ctrl+K</code> (derecha)	40
7.19	Web — Comparativa de precios con cabecera fija y exportación CSV (iz- quierda) y detalle de lista con reordenado <i>drag-and-drop</i> y exportación (de- recha)	40
7.20	Web — Listas (izquierda) y plantillas (derecha) con rejilla responsive	40
7.21	Web — Mapa con panel de tiendas anclado a la derecha (izquierda) y perfil de tienda a dos columnas (derecha)	41
7.22	Web — Tiendas favoritas en rejilla (izquierda) y asistente centrado con copia por mensaje (derecha)	42
7.23	Web — Ruta optimizada con exportación (izquierda) y módulo OCR cen- trado (derecha)	42

7.24 Web — Bandeja de notificaciones (izquierda) y alertas de precio (derecha) centradas	42
7.25 Portal PYME — Página de aterrizaje (<i>landing</i>)	43
7.26 Portal PYME — Inicio de sesión (izquierda) y registro (derecha)	43
7.27 Portal PYME — Onboarding del negocio (alta de datos y CIF)	43
7.28 Portal PYME — Panel de control (<i>dashboard</i>) con estadísticas	44
7.29 Portal PYME — Gestión de productos (izquierda) y carga masiva por CSV (derecha)	45
7.30 Portal PYME — Gestión de precios (tabla editable)	45
7.31 Portal PYME — Gestión de promociones (izquierda) y perfil del negocio (derecha)	45
7.32 Portal PYME — Aprobación de negocios por el administrador	46

CAPÍTULO 1

Introducción y objetivos del proyecto

1.1— Contexto y motivación

El consumidor español afronta la compra cotidiana en un contexto de fuerte variabilidad de precios entre cadenas, ausencia de una fuente unificada y alto coste temporal para comparar ofertas manualmente. Aunque el ahorro potencial entre supermercados es significativo, en la práctica ese ahorro se pierde cuando no se considera el coste de desplazamiento ni el tiempo de ruta.

BargAIn nace para resolver ese problema como una plataforma de compra inteligente que integra precio, distancia y tiempo en una única decisión. El sistema combina una app móvil para consumidores, un portal web para PYMEs, y un backend modular con capacidades geoespaciales, OCR y asistencia conversacional.

1.2— Problema abordado

Las soluciones actuales de comparación de supermercados suelen responder a la pregunta “qué tienda tiene el precio más bajo”, pero no a la pregunta completa “qué compra me conviene realmente según mi ubicación y mis preferencias”.

El problema de BargAIn se formula como una optimización multicriterio en la que intervienen:

- coste total de la cesta,
- distancia adicional de desplazamiento,
- tiempo total estimado de compra y ruta.

1.3— Objetivo principal

Desarrollar una aplicación móvil y web que optimice la cesta de la compra del usuario, calculando la combinación de supermercados y comercios locales que ofrece la mejor relación precio–distancia–tiempo según preferencias configurables.

1.4— Objetivos específicos

1. Implementar una API REST con Django y DRF para usuarios, productos, tiendas, precios, listas, optimización, OCR, asistente y portal business.

2. Integrar una capa geoespacial con PostgreSQL + PostGIS para consultas de proximidad y preparación de rutas.
3. Desarrollar un optimizador multicriterio con OR-Tools y una función de scoring configurable por el usuario.
4. Incorporar OCR con Google Cloud Vision para digitalizar tickets y listas, con validación y corrección manual en frontend.
5. Integrar un asistente LLM (Gemini vía backend proxy) con guardrails de dominio y contexto de compra.
6. Construir un portal para PYMEs con gestión de precios y promociones.
7. Asegurar calidad con tests unitarios, de integración y E2E, además de UAT manual para flujos nativos.
8. Preparar despliegue reproducible en staging con Render, Docker Compose y CI.

1.5— Alcance del proyecto

El alcance del TFG cubre el ciclo completo de ingeniería del software:

- análisis y requisitos,
- diseño arquitectónico y de datos,
- implementación backend y frontend,
- pruebas funcionales y no funcionales,
- despliegue en entorno staging,
- documentación técnica y memoria final.

Quedan fuera del alcance de esta versión: despliegue productivo global, acuerdos formales con todas las cadenas para APIs oficiales, y la versión final de Live Activities nativas en iOS (ADR-010).

1.6— Tecnologías clave

El sistema se apoya en un stack full-stack moderno y coherente:

- **Backend:** Django 5, DRF, Celery, Redis.
- **Datos:** PostgreSQL 16 + PostGIS 3.4.
- **Frontend:** React Native con Expo + companion web en React.
- **IA:** Google Cloud Vision API (OCR) y Gemini vía backend proxy.
- **Optimización:** OR-Tools + cálculo de distancias (OpenRouteService + fallback haversine).
- **Ops:** Docker, GitHub Actions, Render, Sentry.

1.7– Estructura de la memoria

La memoria se organiza para reflejar el recorrido completo del proyecto: contexto y objetivos, estado del arte, comparativa, planificación, requisitos, diseño e implementación, manual de usuario, pruebas y resultados, conclusiones y bibliografía.

Análisis de antecedentes y aportación realizada

2.1— Contexto del sector

El mercado de distribución alimentaria en España presenta una combinación de gran competencia entre cadenas, alta sensibilidad al precio y fuerte fragmentación de oferta. Durante los últimos años, la inflación en alimentación ha incrementado la necesidad de herramientas que permitan comprar mejor sin aumentar el tiempo invertido.

Aunque existen comparadores de precios consolidados, la mayor parte se centra en mostrar información de coste por tienda, sin integrar de forma nativa el impacto logístico de desplazarse a varios establecimientos.

2.2— Estado del arte en comparación de precios

La evolución observada puede resumirse en tres generaciones:

1. **Directorios de precios:** listados estáticos o semiactualizados, con poca cobertura y baja frecuencia de refresco.
2. **Comparadores multisupermercado:** soluciones como Soysuper, Findit u OCU Market, con mejor catálogo y funcionalidades de lista.
3. **Personalización con IA:** tendencia emergente en la que se introducen recomendaciones contextuales y asistencia conversacional.

En España, el salto completo hacia la tercera generación todavía es limitado, especialmente en el segmento de cesta alimentaria diaria.

2.3— Tecnologías relevantes para BargAIn

2.3.1. Geoespacial y rutas

PostGIS permite resolver consultas de proximidad con precisión y rendimiento, mientras que OR-Tools facilita modelar el problema de ordenación de paradas como una variante del TSP/VRP en instancias pequeñas (máximo 3–4 paradas en nuestro caso).

2.3.2. Ingesta de datos de precio

La combinación Scrapy + Playwright es adecuada para catálogos dinámicos de supermercados. Sin embargo, por robustez de producto, BargAIn no depende de una única fuente y combina:

- scraping programado,
- aportaciones de crowdsourcing,
- actualización manual de comercios verificados.

2.3.3. OCR y asistente

Para OCR se adopta Google Cloud Vision API por mejor rendimiento práctico en tickets y fotos reales. Para el asistente se integra Gemini mediante backend proxy, con guardrails para limitar respuestas al dominio de la compra.

2.4— Aportación diferencial de BargAIn

La aportación del proyecto se concreta en cuatro ejes:

- **Optimización multicriterio real** (precio + distancia + tiempo) en lugar de comparación aislada de precios.
- **Inclusión de comercio local** mediante portal business y actualización de precios/promociones.
- **Digitalización de lista** por OCR con validación de usuario.
- **Interacción conversacional** para consultas complejas de compra.

Estas capacidades, integradas en una sola plataforma, posicionan BargAIn en un espacio de mercado poco cubierto por herramientas actuales.

CAPÍTULO 3

Comparación con otras alternativas

3.1– Competidores considerados

Para el análisis competitivo se han considerado cinco soluciones de referencia en el mercado español de comparación de compra:

- Soysuper
- Findit
- OCU Market
- PreciRadar
- RadarPrice

Adicionalmente, se han valorado como competencia indirecta las apps nativas de cadenas y como sustituto no tecnológico la comparación manual con folletos.

3.2– Matriz funcional resumida

Cuadro 3.1: Comparativa de capacidades clave

Capacidad	BargAIn	Soysuper	OCU Market	Resto
Comparación multi-tienda de cesta	Completa	Completa	Parcial	Parcial
Optimización de ruta (precio+dist+tiempo)	Completa	No	No	No
Integración de PY-MEs/comercio local	Completa	No	No	No
OCR de lista/ticket	Completa	No	No	No
Asistente conversacional IA	Completa	No	No	No
Histórico avanzado de precios	Parcial	Limitado	Limitado	Variable

3.3– Brechas estratégicas detectadas

El análisis identifica cuatro brechas de mercado que justifican la propuesta:

1. **Gap precio-ruta:** los comparadores muestran precios, pero no optimizan desplazamiento y tiempo.
2. **Comercio local invisible:** no existe cobertura estable de PYMEs en herramientas generalistas.
3. **Fricción de entrada:** la lista de compra sigue siendo manual en la mayoría de soluciones.
4. **Interacción limitada:** predominan filtros y búsquedas rígidas frente a consultas en lenguaje natural.

3.4– Posicionamiento de BargAIIn

BargAIIn se posiciona como un *optimizador inteligente de compra* y no solo como comparador. Su propuesta de valor combina ahorro económico, eficiencia logística y experiencia de usuario asistida por IA.

3.5– Riesgos competitivos

Los riesgos principales identificados son:

- reacción de competidores consolidados con mayor base de usuarios,
- dependencia de fuentes de datos de terceros para scraping,
- necesidad de mantener alta calidad de datos para sostener confianza.

Como mitigación, se adopta una estrategia de fuentes híbridas de datos, modularidad arquitectónica y pipeline de validación continua.

CAPÍTULO 4

Análisis temporal y costes de desarrollo

4.1— Planificación temporal

La planificación de BargAI_n se definió con una dedicación planificada de 300 horas, distribuidas en fases funcionales y técnicas, que se amplió hasta 325 horas reales al incorporar el cierre documental y la preparación de la defensa. El proyecto se ejecutó de forma iterativa, manteniendo trazabilidad en `TASKS.md` y en el árbol `.planning/`.

Cuadro 4.1: Resumen por fases del proyecto

Fase	Semanas	Horas est.	Estado
F1 Análisis y diseño	S1–S3	45	Completada
F2 Infraestructura base	S3–S5	30	Completada
F3 Desarrollo core backend	S5–S12	95	Completada
F4 Desarrollo frontend	S8–S16	70	Completada
F5 IA, optimizador y scraping	S10–S17	35	Completada
F6 Portal business y app móvil	S15–S18	25	Completada
F7 Pruebas, deploy y cierre	S18–S20	25	Completada
Total		325	Cerrado

4.2— Metodología de trabajo

Se aplicó una metodología incremental con ciclos cortos por fase:

- definición de alcance y criterios de aceptación,
- implementación por módulos,
- validación automatizada (lint + tests),
- verificación funcional y documental,
- cierre de fase con artefactos de evidencia.

Este enfoque permitió absorber cambios sin comprometer la estabilidad del sistema.

Cuadro 4.2: Coste estimado del desarrollo

Bloque	Horas	Coste (EUR)
Análisis y diseño	45	1.490,40
Infraestructura	30	861,60
Backend + frontend	165	4.738,80
IA, optimizador y scraping	35	1.005,20
Portal business y app móvil	25	718,00
Pruebas y despliegue	25	594,25
Total	325	9.408,25

4.3– Estimación de costes

La estimación económica se realizó mediante coste-hora por rol junior, alineada con referencias del informe de perfiles TIC de la Junta de Andalucía (2018).

Adicionalmente, el coste operativo en staging se mantuvo bajo durante el TFG gracias al uso de planes gratuitos o de bajo consumo (Render, Sentry, GitHub Actions).

4.4– Riesgos y mitigaciones

Los riesgos operativos principales fueron:

- **Cambios en fuentes de scraping:** mitigado con spiders modulares y fallback de crowdsourcing.
- **Dependencia de APIs externas:** mitigado con caché y degradación controlada.
- **Carga de trabajo en fases finales:** mitigado con cierre incremental de entregables y automatización de validación.

4.5– Resultado de planificación

La planificación permitió completar el alcance funcional priorizado del TFG, incluyendo arquitectura, implementación, pruebas y despliegue de staging, manteniendo coherencia entre roadmap, requisitos y evidencias de verificación.

Análisis de requisitos

5.1— Actores del sistema

El sistema define cuatro actores principales:

- **Consumidor:** crea listas, compara precios, optimiza rutas, usa OCR y consulta al asistente.
- **Comercio/PYME:** gestiona precios y promociones desde el portal business.
- **Administrador:** verifica perfiles, modera datos y supervisa la operación.
- **Sistema automatizado:** tareas de scraping y mantenimiento en segundo plano.

5.2— Requisitos de información

El modelo de información se articula en doce bloques (RI-001 a RI-012), que cubren:

- usuarios y preferencias de optimización,
- catálogo (productos, categorías, marcas),
- tiendas geolocalizadas y cadenas,
- precios actuales e históricos,
- listas de compra y resultados de optimización,
- perfiles business y promociones,
- sesiones OCR y conversaciones del asistente.

5.3— Requisitos funcionales

Se definieron 35 requisitos funcionales (RF-001 a RF-035) agrupados por dominios:

Cuadro 5.1: Agrupación de requisitos funcionales

Dominio	IDs	Capacidad principal
Autenticación y perfil	RF-001..RF-005	Registro, login JWT, preferencias
Catálogo de productos	RF-006..RF-010	Búsqueda, detalle, crowdsourcing
Tiendas y mapa	RF-011..RF-014	Proximidad, detalle, favoritos
Precios	RF-015..RF-019	Comparación, histórico, alertas
Listas de compra	RF-020..RF-023	CRUD, compartir, plantillas
Optimizador	RF-024..RF-027	Ruta multicriterio y recálculo
OCR	RF-028..RF-029	Captura y matching fuzzy
Asistente LLM	RF-030..RF-031	Consultas naturales y sugerencias
Portal business	RF-032..RF-034	Onboarding, precios, promociones
Notificaciones	RF-035	Push/email por eventos

5.4— Criterios de aceptación del producto

Los criterios de aceptación se validan a nivel funcional y de experiencia:

- flujo de alta y acceso operativo en menos de 3 minutos,
- comparación de cesta completa en entorno de tiendas cercanas,
- optimización con top-3 rutas y desglose por parada,
- OCR con confirmación/edición previa a la inserción en lista,
- portal PYME con gestión de precios y promociones,
- notificaciones de eventos críticos (alertas, promociones, cambios compartidos).

5.5— Requisitos no funcionales

Se definieron ocho requisitos no funcionales (RNF-001 a RNF-008):

- rendimiento API (p95 bajo para operaciones CRUD y optimización),
- disponibilidad de staging,
- seguridad (JWT, HTTPS, rate limiting),
- usabilidad (flujo principal corto y criterios de accesibilidad),
- escalabilidad arquitectónica,
- mantenibilidad con cobertura y convenciones,
- compatibilidad multiplataforma,
- cumplimiento RGPD.

5.6— Reglas de negocio

Las reglas RN-001..RN-010 gobiernan límites y políticas operativas:

- radio máximo de búsqueda y número máximo de paradas,
- caducidad y prioridad de fuentes de precio,
- restricciones del asistente al dominio de compra,
- verificación previa de perfiles business,
- políticas de scraping responsable y frecuencia de ejecución.

5.7— Diagramas de casos de uso

Los diagramas de casos de uso recogen las interacciones principales de cada actor con el sistema.

5.8— Trazabilidad

La trazabilidad entre requisitos, implementación y pruebas se mantiene en:

- docs/memoria/07-requisitos.md (especificación completa),
- TASKS.md (estado por fase/tarea),
- tests/unit, tests/integration y frontend/web/e2e (verificación ejecutable).

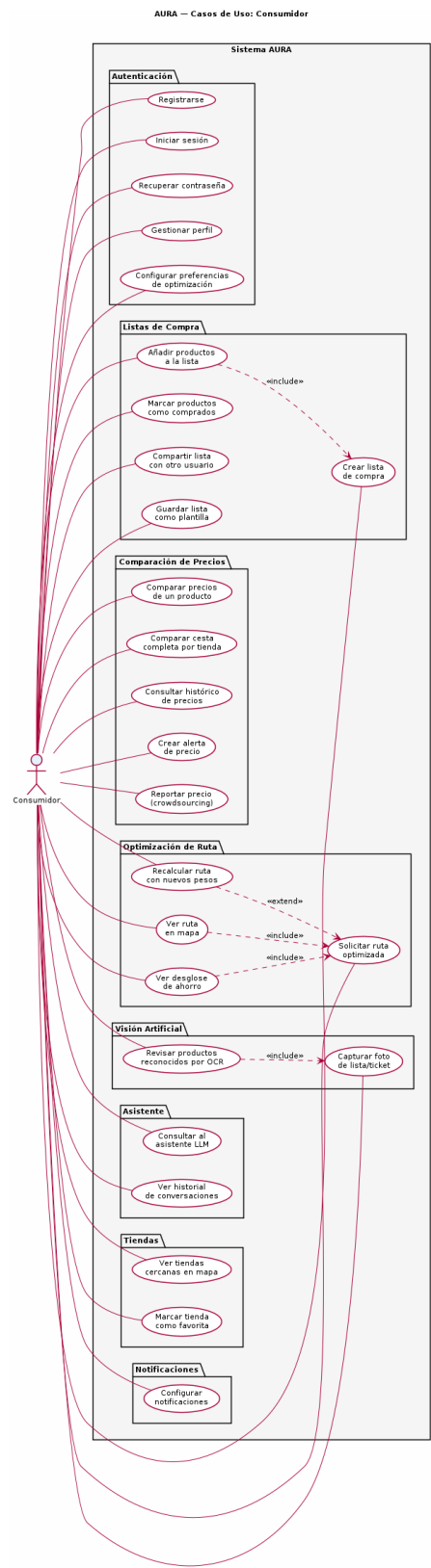


Figura 5.1: Casos de uso del Consumidor

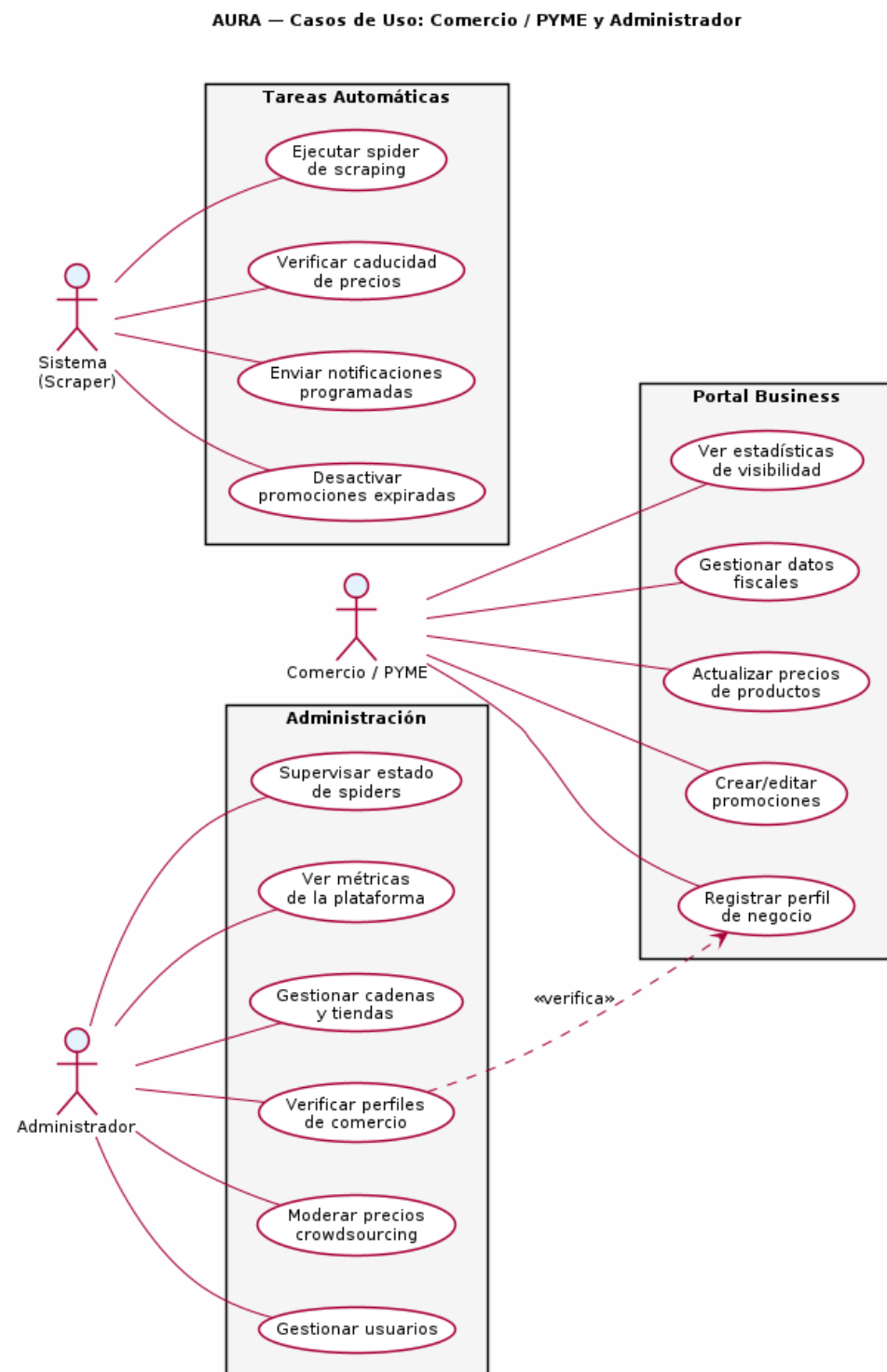


Figura 5.2: Casos de uso del Comercio y Administrador

Diseño e Implementación

6.1— Arquitectura del Sistema

6.1.1. Visión general

BargAIn sigue una arquitectura de **tres capas** clásica (presentación, lógica de negocio y datos), desplegada mediante contenedores Docker y organizada internamente según los principios de la **arquitectura hexagonal** (puertos y adaptadores). Esta elección permite desacoplar la lógica de dominio de los detalles de infraestructura — bases de datos, APIs externas, brokers de mensajería — facilitando el testing independiente de cada componente.

La arquitectura global se divide en cinco bloques principales:

1. **Aplicación cliente:** React Native (Expo) para iOS y Android, con una aplicación web React para el Portal Business de PYMEs.
2. **API Backend:** Django REST Framework como núcleo de la lógica de negocio, expuesto bajo HTTPS mediante Gunicorn + Nginx.
3. **Capa de datos:** PostgreSQL 16 con extensión PostGIS 3.4 para consultas geoespaciales.
4. **Capa de tareas asíncronas:** Celery con Redis como broker, para scraping periódico, envío de notificaciones, procesamiento OCR y cálculos del optimizador.
5. **Integraciones y servicios auxiliares:** Google Gemini API (asistente LLM), OpenRouteService (cálculo de distancias reales), Google Cloud Vision API (OCR) y Sentry (monitorización de errores).

6.1.2. Patrón de organización interna del backend

Cada aplicación Django del backend sigue una estructura modular consistente:

```
apps/<modulo>/
|-- models.py          # Entidades de dominio (ORM)
|-- serializers.py      # Transformacion de datos (entrada/salida)
|-- views.py           # ViewSets y vistas (controladores)
|-- urls.py            # Rutas del modulo
|-- services.py         # Logica de negocio
|-- tasks.py           # Tareas Celery asincronas
```

```
-- permissions.py      # Logica de autorizacion
-- tests/
```

La separación entre `views.py` y `services.py` es deliberada: los ViewSets se limitan a la gestión HTTP (autenticación, serialización, códigos de respuesta), mientras que la lógica de negocio reside exclusivamente en los servicios, lo que permite invocarla desde tareas Celery o tests sin simular peticiones HTTP.

La Figura 6.1 resume la distribución por capas de la solución.

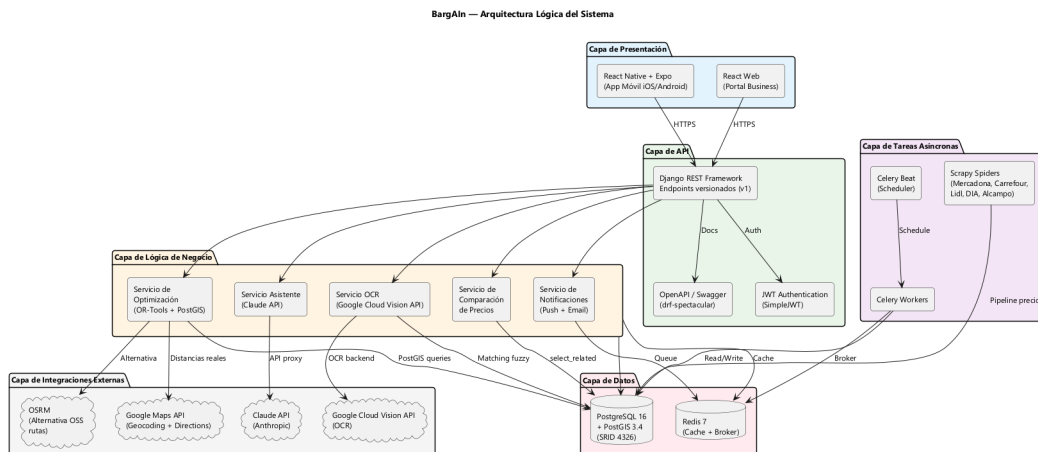


Figura 6.1: Arquitectura por capas de BargAI

6.1.3. Comunicación entre componentes

- **Frontend → Backend:** API REST con autenticación JWT. Axios gestiona la renovación automática del token de acceso ante un 401 Unauthorized.
- **Backend → Celery:** Encolar tareas mediante `.delay()` o `.apply_async()` sobre el broker Redis.
- **Backend → PostgreSQL:** Django ORM con consultas geoespaciales mediante `django.contrib.gis`.
- **Backend → APIs externas:** Clientes HTTP aislados en la capa de servicios de cada módulo (p.ej. `apps/optimizer/services/distance.py`) para Google Places, Gemini API y OpenRouteService.

6.2— Modelo de Datos

6.2.1. Módulo de usuarios

El modelo `User` extiende `AbstractBaseUser` de Django para permitir el login mediante email. Se definen tres roles: `consumer`, `business` y `admin`. El campo `default_location` utiliza `PointField(SRID=4326)`.

Las preferencias de optimización (`w_precio`, `w_distancia`, `w_tiempo`) se almacenan como decimales con restricción de que su suma sea igual a 1.0, validada a nivel de serializer:

```
def validate(self, data: dict) -> dict:
    weights = [
```

```

        data.get('weight_price', 0),
        data.get('weight_distance', 0),
        data.get('weight_time', 0),
    ]
    if abs(sum(weights) - 1.0) > 0.001:
        raise serializers.ValidationError(
            "Los pesos de optimizacion deben sumar exactamente 1.0"
        )
    return data

```

6.2.2. Módulo de productos

La búsqueda de productos por nombre utiliza **matching fuzzy** mediante la extensión PostgreSQL `pg_trgm` (trigramas). El campo `normalized_name` almacena el nombre normalizado (minúsculas, sin tildes, sin caracteres especiales) con índices GIN de trigramas:

```

def search_products(query: str, limit: int = 20) -> QuerySet[Product]:
    return (
        Product.objects
        .annotate(similarity=TrigramSimilarity(
            'normalized_name', normalize(query)))
        .filter(similarity__gt=0.3, is_active=True)
        .order_by('-similarity')[:limit]
    )

```

6.2.3. Módulo de tiendas

El campo `location` de `Store` es un `PointField` con índice GiST para búsquedas geoespaciales eficientes:

```

def get_stores_in_radius(
    lat: float, lng: float, radius_km: float
) -> QuerySet[Store]:
    user_location = Point(lng, lat, srid=4326)
    return (
        Store.objects
        .filter(
            location__dwithin=(user_location, D(km=radius_km)),
            is_active=True
        )
        .annotate(distance=Distance('location', user_location))
        .order_by('distance')
    )

```

El uso de `ST_DWithin` en lugar de `ST_Distance` en el `WHERE` es deliberado: `ST_DWithin` puede aprovechar el índice GiST, mientras que calcular distancias en el filtro forzaría un escaneo completo de la tabla.

6.2.4. Módulo de precios

El histórico de precios se materializa automáticamente: cada precio es inmutable una vez creado. La actualización de un precio supone la inserción de un nuevo registro. Una tarea Celery periódica (`expire_stale_prices`) marca como expirados los precios de scraping con más de 48 horas de antigüedad y los de crowdsourcing con más de 24 horas.

6.2.5. Módulo del optimizador

El módulo del optimizador incluye tres entidades relacionales para almacenar el resultado de la optimización de forma trazable:

- **OptimizationResult:** resultado global con pesos, modo y métricas totales. Incluye `route_data` JSONB para compatibilidad de lectura rápida.
- **OptimizationRouteStop:** cada parada de la ruta con tienda, orden, distancia, tiempo y subtotal.
- **OptimizationRouteStopItem:** ítem concreto por parada, con producto resuelto, precio elegido, texto original y puntuación de similitud.

6.2.6. Índices clave

Cuadro 6.1: Índices principales de la base de datos

Tabla	Columna(s)	Tipo	Justificación
store	location	GiST	Búsquedas geospaciales por radio
price	(product, store, created_at)	B-Tree	Histórico y precio actual
price	expires_at	B-Tree parcial	Filtrar precios vigentes
product	normalized_name	GIN (trigram)	Búsqueda fuzzy por nombre
shopping_list_item	(shopping_list, is_checked)	B-Tree	Ítems pendientes

6.2.7. Modelo entidad-relación

La Figura 6.2 muestra el diagrama entidad-relación completo de la base de datos.

6.2.8. Diagrama de clases

La Figura 6.3 detalla las clases principales del dominio y sus relaciones.

6.3– Diseño de la API REST

6.3.1. Estructura general

La API sigue las convenciones REST y está versionada bajo el prefijo `/api/v1/`. Todos los endpoints requieren autenticación JWT salvo los de registro y login. Las respuestas siguen el formato unificado:

```
// Respuesta exitosa
{
  "success": true,
  "data": { ... },
  "meta": { "count": 42, "page": 1, "page_size": 20 }
}

// Respuesta de error
{
  "success": false,
  "error": {
```

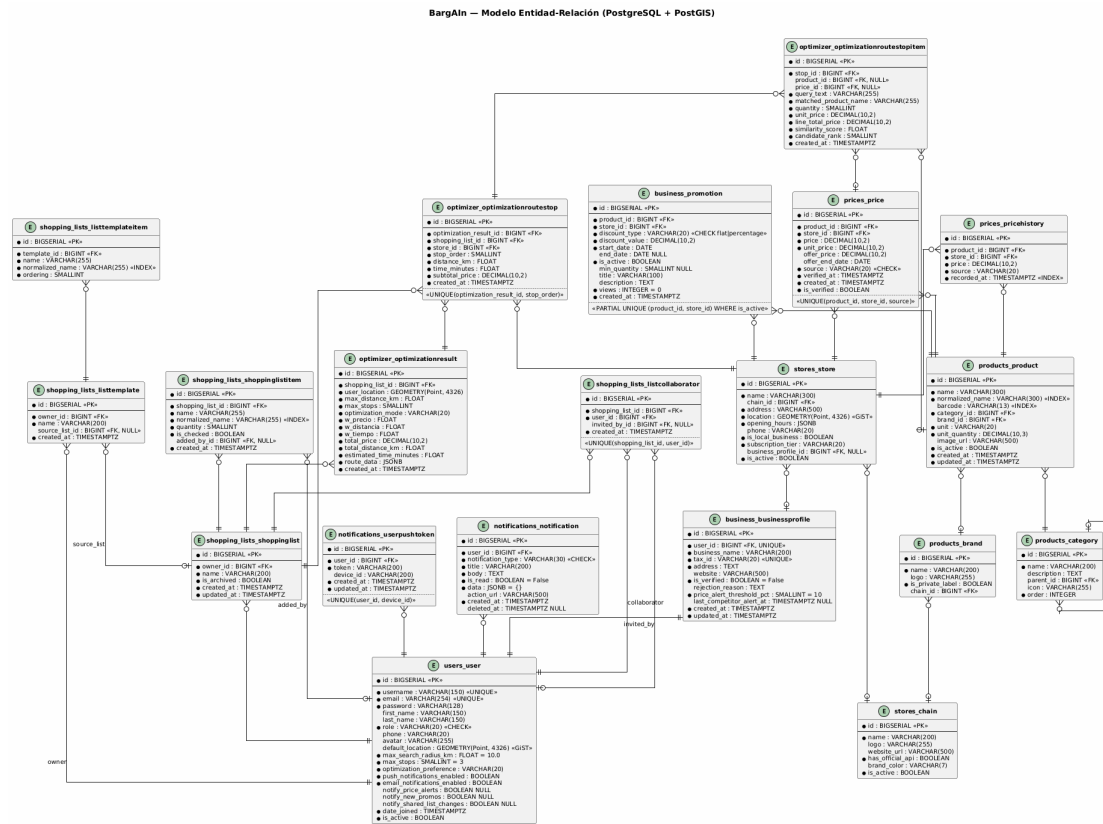


Figura 6.2: Diagrama entidad-relación de BargAIn

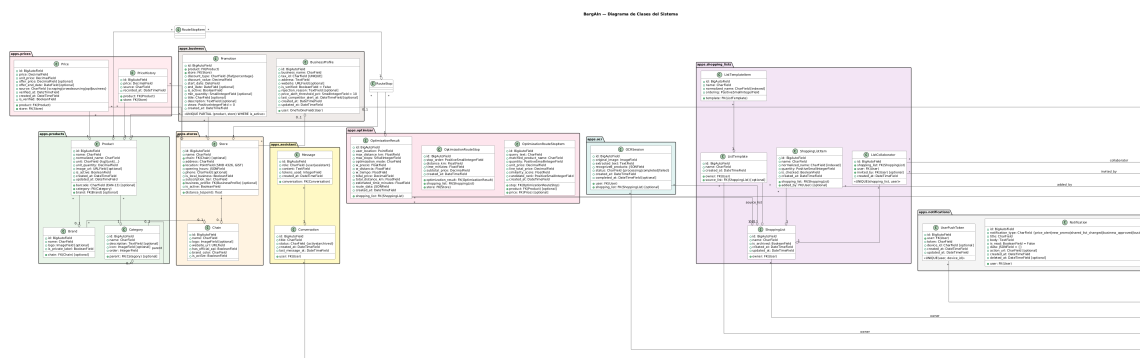


Figura 6.3: Diagrama de clases del dominio

```

    "code": "STORE_NOT_FOUND",
    "message": "No se encontraron tiendas en el radio especificado",
    "details": {}
  }
}

```

6.3.2. Endpoints principales

Cuadro 6.2: Endpoints de autenticación y usuarios

Método	Endpoint	Descripción
POST	/api/v1/auth/register/	Registro de nuevo usuario
POST	/api/v1/auth/login/	Login: access + refresh token
POST	/api/v1/auth/token/refresh/	Renovar access token
POST	/api/v1/auth/logout/	Invaldar refresh token
GET/PATCH	/api/v1/users/me/	Perfil del usuario autenticado
GET/PATCH	/api/v1/users/me/preferences/	Preferencias de optimización

Cuadro 6.3: Endpoints del optimizador, OCR y asistente

Método	Endpoint	Descripción
POST	/api/v1/optimize/	Calcular o recalcular ruta óptima
GET	/api/v1/optimize/?shopping_list_id={id}	Calcular última ruta persistida
POST	/api/v1/ocr/scan/	Procesar imagen y devolver ítems reconocidos
GET/POST	/api/v1/assistant/conversations/	Listar y crear conversaciones
POST	/api/v1/assistant/conversations/{id}/messages/	Enviar mensajes al asistente

6.3.3. Autenticación y autorización

Se implementa un sistema de permisos a tres niveles:

- **IsAuthenticated:** Aplicado por defecto a todos los endpoints privados.
- **IsBusinessOwner:** Permite operar solo sobre el perfil de negocio propio.
- **IsAdminUser:** Para endpoints de administración del sistema.
- **IsShoppingListOwnerOrShared:** Permite acceso al propietario y colaboradores invitados.

Los tokens JWT se generan con `django-rest-framework-simplejwt`. El token de acceso tiene validez de 60 minutos y el de refresco de 7 días con rotación automática.

6.4— Implementación del Backend

6.4.1. Módulo de notificaciones

El módulo gestiona el buzón de notificaciones del usuario (inbox) y el envío de notificaciones push mediante **Expo Push Notifications**. Las entregas se procesan asincrónicamente con tareas Celery.

- **Notification:** Registro persistente con `notification_type` (`price_alert`, `new_promo`, `shared_list_changed`, `business_approved`, `business_rejected`), campos `title/body`, bandera `is_read`, data JSONB para payload variable, `action_url` para deep links y `deleted_at` para soft-delete.
- **UserPushToken:** Token Expo registrado por dispositivo. La clave única (`user`, `device_id`) evita duplicados al reinstalar la app.

6.4.2. Módulo de portal business

- **BusinessProfile:** Perfil verificable con `tax_id` (CIF/NIF único), estado de verificación y umbral de alerta de competidor (`price_alert_threshold_pct`).
- **Promotion:** Promoción temporal de un producto en una tienda. Restricción parcial única (`product`, `store`) WHERE `is_active` garantiza que no existan dos promociones activas para el mismo par.

La tarea Celery `check_competitor_prices` se ejecuta cada 6 horas y alerta al propietario cuando detecta diferencias superiores al umbral configurado.

6.5— El Algoritmo de Optimización de Rutas

6.5.1. Formulación del problema

El algoritmo de optimización es la aportación técnica central del TFG. El problema es una variante del **Vehicle Routing Problem (VRP)** con objetivos múltiples, resuelto mediante una solución heurística eficiente.

Entrada:

- Lista de la compra $L = \{i_1, i_2, \dots, i_n\}$ (n ítems textuales)
- Ubicación del usuario $u = (lat, lng)$
- Radio máximo r (km), número máximo de paradas k (1–4)
- Pesos de preferencia w_{precio} , $w_{distancia}$, w_{tiempo} (suman 1.0)

Salida: Top-3 asignaciones de ítems resueltos a tiendas que minimizan la función de coste.

6.5.2. Función de scoring multicriterio

$$\text{Score}(A) = w_{precio} \cdot \text{ahorro_norm}(A) - w_{distancia} \cdot \text{dist_extra_norm}(A) - w_{tiempo} \cdot \text{tiempo_extra_norm}(A) \quad (6.1)$$

Donde $\text{ahorro_normalizado} = (\text{coste_single_best} - \text{coste}(A)) / \text{coste_single_best} \in [0, 1]$ y $\text{distancia_extra_normalizada} = \max(0, \text{dist_ruta}(A) - \text{dist_single}) / (r \times 1,5)$.

6.5.3. Pipeline de ejecución

El algoritmo se ejecuta en seis fases secuenciales dentro de `apps/optimizer/services.py`:

1. **Fase 0 — Resolución textual de ítems:** Preselección por trigramas sobre `Product.normalized_name` ranking fuzzy (`token_set_ratio`), y validación contextual por disponibilidad de precio.

2. **Fase 1 — Ingesta de precios candidatos:** Para cada producto resuelto, se obtienen los precios vigentes en tiendas dentro del radio con una sola consulta optimizada con `select_related` y filtros geoespaciales.
3. **Fase 2 — Pre-filtrado de tiendas:** Se limita a 30 tiendas candidatas máximo para controlar la explosión combinatoria ($C(30, 3) = 4.060$ combinaciones para $k=3$).
4. **Fase 3 — Generación y evaluación de asignaciones:** Para cada combinación de tiendas, asignación greedy óptima de ítems al menor precio disponible.
5. **Fase 4 — Cálculo de distancias reales:** Matriz de distancias vía **OpenRoute-Service (ORS)** en una sola petición. Fallback haversine si ORS no está disponible:

```
POST https://api.openrouteservice.org/v2/matrix/driving-car
Authorization: {ORS_API_KEY}

{
  "locations": [[lng1, lat1], [lng2, lat2], ...],
  "metrics": ["distance", "duration"],
  "units": "km"
}
```

6. **Fase 5 — Scoring y ranking:** Normalización de valores al rango $[0, 1]$ y devolución de las top-3 asignaciones ordenadas por score descendente.

6.5.4. Diagrama de secuencia del optimizador

La Figura 6.4 ilustra el flujo de mensajes entre componentes durante una petición de optimización.

6.5.5. Integración con OR-Tools

Para combinaciones complejas donde el problema de asignación supera la heurística greedy, se utiliza **OR-Tools de Google** para resolver el problema como programación lineal entera (ILP), garantizando la optimalidad de la asignación de productos a tiendas dentro de un subconjunto fijo de tiendas.

6.5.6. Complejidad y rendimiento

Cuadro 6.4: Tiempos estimados de ejecución del optimizador

Escenario	Tiendas	Combinaciones (k=3)	Tiempo
Lista pequeña (5 prod.)	≤ 15	455	< 1 s
Lista media (20 prod.)	≤ 30	4.060	< 3 s
Lista grande (50 prod.)	≤ 30	4.060	< 5 s

El umbral de 5 s (RNF-001) se cumple gracias a: pre-filtrado de tiendas, caché de matriz de distancias en Redis, y ejecución como tarea Celery asíncrona.

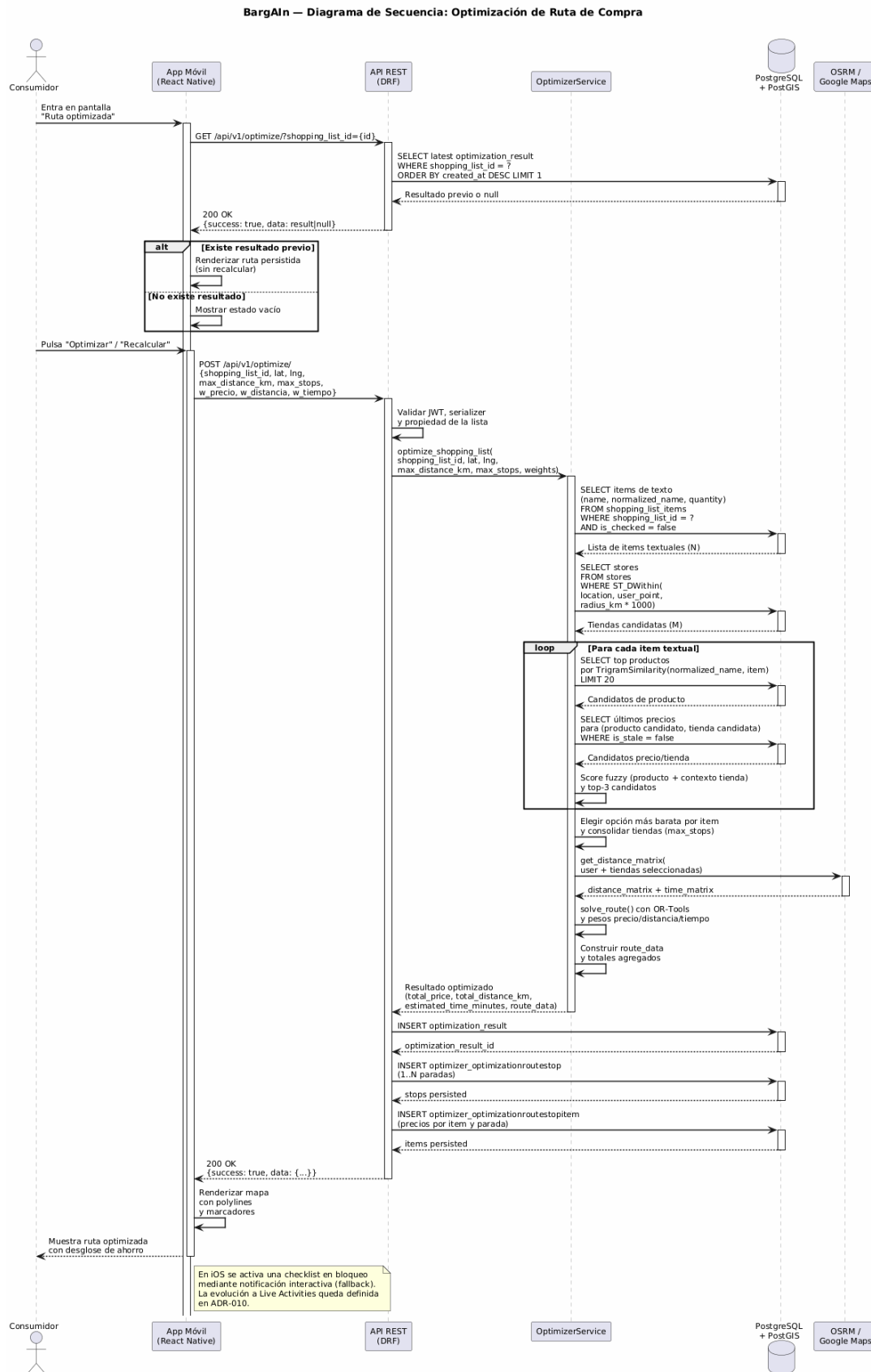


Figura 6.4: Diagrama de secuencia del proceso de optimización de ruta

6.6— Módulo de Web Scraping

El scraper es un **proyecto Scrapy independiente** (`scraping/`) desacoplado del backend Django, que comunica con la base de datos a través de la API REST del propio backend.

Cada cadena requiere una estrategia diferente:

- **Mercadona:** API interna (`tienda.mercadona.es/api/`) no oficial pero estable, accesible mediante peticiones HTTP estándar.
- **Carrefour / Lidl / DIA:** Catálogos con renderizado del lado del cliente (SPA), por lo que se usa **Playwright** para ejecutar el JavaScript y obtener el DOM hidratado.

Los spiders se programan con **Celery Beat**, con un delay mínimo de 2 segundos entre peticiones consecutivas (regla de negocio RN-010).

6.7— Módulo OCR

El backend invoca **Google Cloud Vision API** con la imagen capturada por el usuario. La decisión de migrar desde Tesseract a Vision API se adoptó tras comprobar que Tesseract no reconoce con suficiente claridad tickets y listas fotografiadas en condiciones reales:

```
from google.cloud import vision

def extract_text_from_image(image_bytes: bytes) -> list[str]:
    """Extrae líneas de texto mediante Google Cloud Vision API."""
    client = vision.ImageAnnotatorClient()
    image = vision.Image(content=image_bytes)
    response = client.text_detection(image=image)
    full_text = response.full_text_annotation.text
    return [line.strip() for line in full_text.splitlines() if line.strip()]
```

Cada línea extraída se normaliza y se busca en el catálogo mediante el servicio de matching fuzzy de `apps/products`. Se devuelve una lista de candidatos con nivel de confianza.

6.7.1. Diagrama de secuencia OCR

La Figura 6.5 muestra el flujo de procesamiento desde la captura de imagen hasta la inserción de ítems en la lista.

6.8— Integración del Asistente LLM

El asistente conversacional utiliza la **Google Gemini API** (`gemini-2.0-flash-lite`) como modelo subyacente (ADR-008). El backend actúa como proxy, añadiendo contexto de la compra del usuario al system prompt antes de enviar cada mensaje a la API mediante el SDK `google-genai`.

Se aplica una **ventana deslizante** sobre el historial de mensajes (últimos 20 mensajes + system prompt) para mantener el consumo de tokens controlado. El system prompt incluye un guardrail que limita las respuestas a consultas de compra doméstica (RN-007).

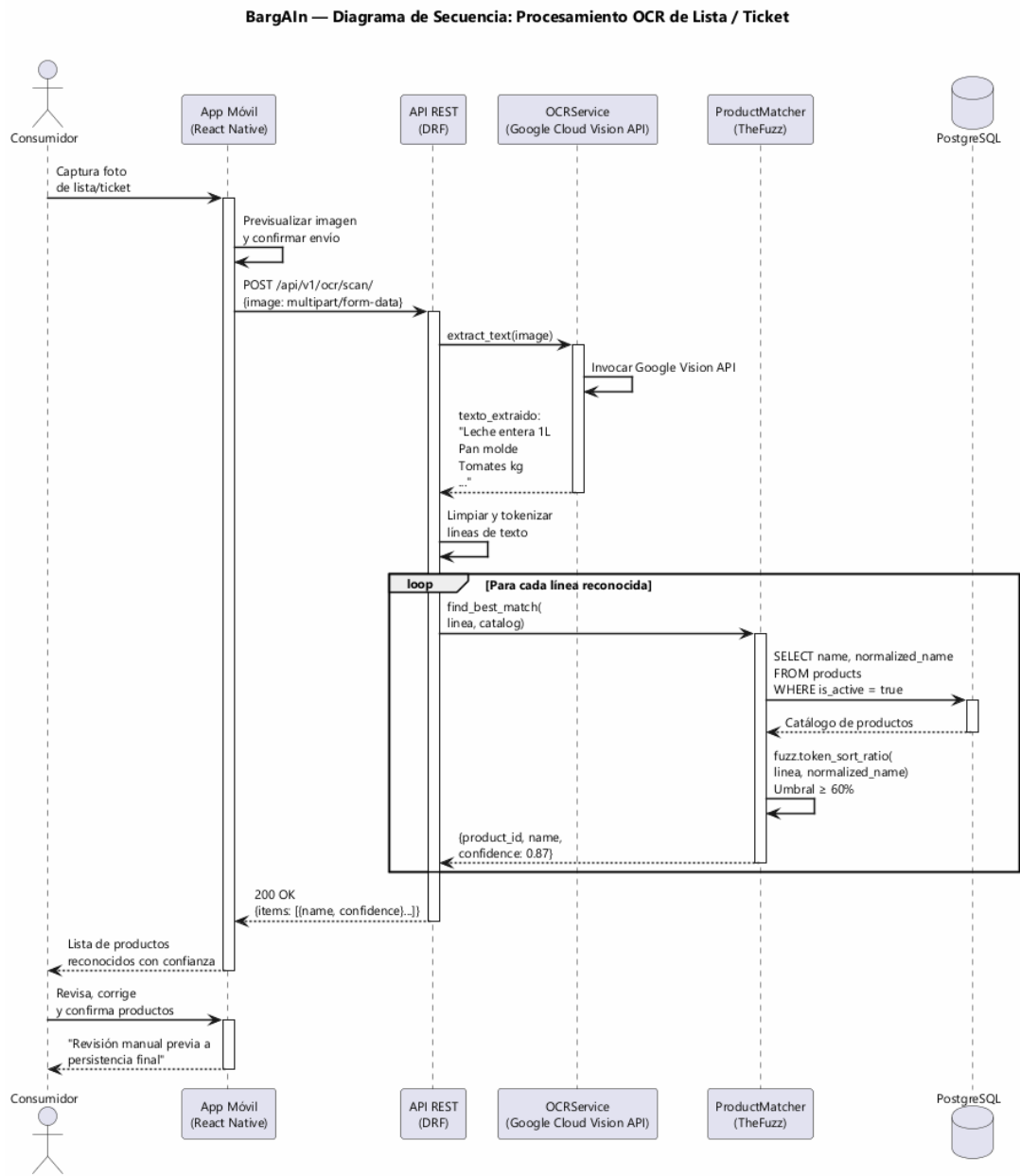


Figura 6.5: Diagrama de secuencia del flujo OCR

6.9— Infraestructura y Despliegue

6.9.1. Contenedores Docker

Cada componente se ejecuta en un contenedor independiente, orquestados con Docker Compose:

- **nginx:** Reverse proxy y servicio de estáticos.
- **backend:** Django + Gunicorn (3 workers).
- **celery-worker:** Workers para tareas asíncronas.
- **celery-beat:** Scheduler de tareas periódicas.
- **db:** PostgreSQL 16 + PostGIS.
- **redis:** Broker Celery + caché.

Para el entorno de desarrollo se usa el modelo híbrido (ADR-002): backend en Docker, frontend nativo en el host. Docker volumes en Windows rompen el HMR de Metro/Expo, por lo que el frontend no se dockeriza.

6.9.2. Pipeline CI/CD

El repositorio incluye tres workflows de GitHub Actions:

- **ci-backend.yml:** Ejecuta `ruff check`, `ruff format --check` y `pytest --cov` en cada push a cualquier rama.
- **ci-frontend.yml:** Ejecuta `eslint`, `prettier --check` y `jest --coverage` en cada push.
- **ios-build.yml:** Genera el `.ipa` unsigned mediante `expo prebuild + xcodebuild` en un runner `macos-latest`. El artefacto se descarga y se instala en el iPhone de demo con Sideloadly.

El deploy a staging se realiza mediante **Render Blueprints**. El repositorio define dos variantes declarativas: `render.yaml`, con web service, worker Celery, scheduler beat, Redis y base de datos PostgreSQL independientes; y `render.free.yaml`, la variante free tier empleada durante el TFG, que consolida Gunicorn y un proceso Celery worker + beat embebido en un único web service. Cada push a `main` dispara un redespiegue automático.

6.9.3. Variables de entorno

La configuración sensible (claves API, credenciales de BD, secret key de Django) se gestiona exclusivamente mediante variables de entorno, nunca hardcodeadas. Se proporciona `.env.example` con los nombres de todas las variables requeridas.

6.10— Diseño de la Interfaz de Usuario

6.10.1. Sistema de diseño

El frontend define un sistema de diseño propio en `src/theme/` con los siguientes tokens:

Tipografía: Inter (sans-serif) con escala modular. **Espaciado:** Escala de 4px.

Cuadro 6.5: Paleta de colores del sistema de diseño

Token	Valor	Uso
primary	#2E7D32	Acciones principales, CTAs
primaryLight	#60AD5E	Estados hover, fondos sutiles
secondary	#FF6F00	Ahorros, destacados, ofertas
surface	#FAFAFA	Fondo de tarjetas
error	#C62828	Errores y alertas
textPrimary	#212121	Texto principal
textSecondary	#757575	Texto de apoyo

6.10.2. Pantallas principales

La aplicación se estructura en cinco pestañas principales:

1. **Inicio / Dashboard:** Resumen de la lista activa, precio estimado en la tienda más barata cercana, acceso rápido al asistente y alertas activas.
2. **Mi Lista:** Gestión de la lista activa con buscador de autocompletado, grupos por categoría, swipe-to-delete/check y botón de optimizar ruta.
3. **Explorar:** Mapa interactivo con marcadores de tiendas y panel inferior deslizable con detalles de la tienda seleccionada.
4. **Optimizador:** Sliders de pesos, cards comparativas de las top-3 rutas y visualización de la ruta en mapa con polylines y marcadores numerados.
5. **Perfil:** Datos personales, preferencias de optimización, configuración de notificaciones y acceso al historial del asistente.

6.10.3. Adaptación a uso web (responsive y conveniencias de escritorio)

La misma base de código Expo (`frontend/src`) se reutiliza para ejecución en navegador de escritorio mediante `react-native-web`, sin duplicar pantallas ni alterar la navegación. La adaptación a escritorio se implementa como una capa transversal sobre las 15 pantallas existentes, organizada en cuatro ejes:

1. **Diseño responsive.** El hook `useBreakpoint()` (`src/hooks/`) clasifica el ancho del viewport en `mobile` ($<768\text{px}$), `tablet` ($768\text{--}1023\text{px}$) y `desktop` ($\geq 1024\text{px}$). Las pantallas centran su contenido a un ancho máximo legible (entre 560 y 900 px según la pantalla) y los listados de tarjetas pasan de 1 columna en móvil a 2–4 columnas en escritorio. El componente `MasterDetailLayout` aporta una disposición maestro-detalle de dos paneles, y la barra de pestañas inferior se centra a 900 px en escritorio.
2. **Interacción con ratón y teclado.** Se añaden estados *hover* (tintes y realces), atajos de teclado (`Cmd/Ctrl+K` para enfocar el buscador, `Enter` para añadir ítem, `Esc` para cerrar), *auto-focus* de campos al montar y *tooltips* web (`WebTooltip`). El reordenado de ítems de una lista emplea *drag-and-drop* HTML5 nativo en la variante `ListDetailScreen.web.tsx`.
3. **Conveniencias de escritorio.** Las utilidades de `src/utils/webExport.ts` permiten exportar la lista de la compra a `.txt`, la comparativa de precios a `.csv` y la ruta a fichero, así como copiar al portapapeles direcciones y enlaces. La generación de CSV incluye un *guard* de inyección de fórmulas (OWASP) que escapa las celdas que comienzan por `=`, `+`, `-` o `@`.

4. **Deep-linking.** La configuración `src/navigation/linking.ts` registra URLs estables para las pantallas (p.ej. `/app/lists/:listId`, `/app/map/store/:storeId`, `/app/home/catalog?q=...`), de modo que el estado de la aplicación es enlazable y navegable desde la barra de direcciones del navegador.

Todas las adiciones están condicionadas a la plataforma web (comprobación de `Platform.OS`) o al *breakpoint* activo, de forma que el comportamiento en móvil permanece inalterado. Estas mejoras se incorporaron en una fase específica de adaptación web posterior al cierre del plan inicial, y se validaron con 111 pruebas Jest (16 *suites*) y una batería de verificación manual en navegador.

Manual de Usuario

7.1— Introducción

BargAIn permite al usuario gestionar su compra diaria desde una app móvil (iOS y Android) y un portal web para PYMEs, comparar precios entre supermercados cercanos, y calcular la ruta de compra más eficiente según sus preferencias de precio, distancia y tiempo.

7.2— Requisitos de uso

- Dispositivo móvil iOS (14+) o Android (10+) con la app instalada, o navegador web moderno para el portal business.
- Conexión a Internet.
- Cuenta de usuario registrada en el sistema.
- Permisos de ubicación concedidos a la app (para funciones de mapa y optimización).

Para el entorno de desarrollo local:

- Backend levantado en Docker (`make dev`).
- Base de datos migrada (`make migrate-docker`).
- Frontend Expo en host (`make frontend`).

7.3— Flujo de acceso

1. Abrir la app en Expo/Web.
2. Seleccionar “Iniciar sesión” ó “Registrarse”.
3. Introducir credenciales válidas.
4. Tras autenticación, se carga la navegación principal por pestañas.

El sistema usa JWT con refresco automático. Si el access token expira, la sesión se recupera sin intervención del usuario cuando el refresh token sigue siendo válido (7 días desde el último uso).

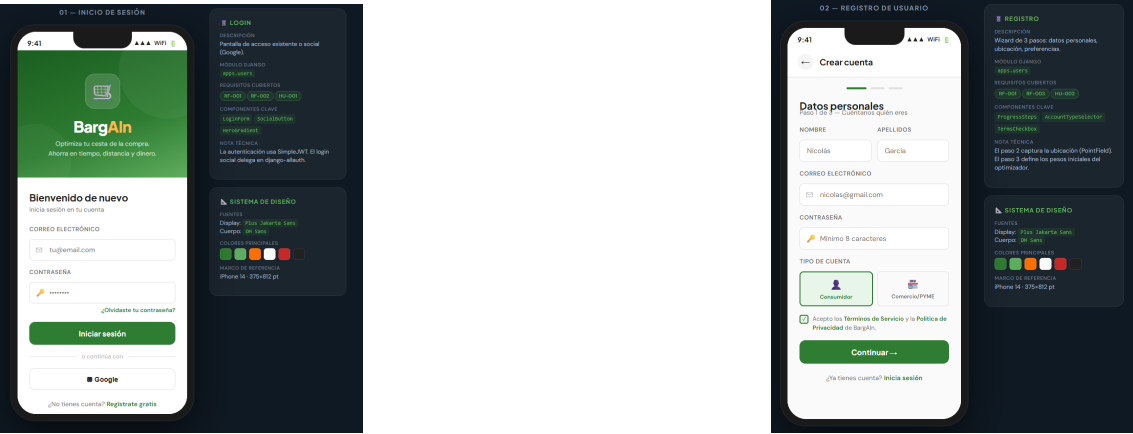


Figura 7.1: Pantallas de inicio de sesión y registro

7.3.1. Pantallas de acceso

7.4— Pantalla principal (Home)

La pantalla de inicio centraliza la información más relevante del usuario:

- Resumen de listas activas con el precio estimado en la tienda más barata cercana.
- CTA para crear lista cuando el usuario no tiene listas activas.
- Buscador global para navegar rápidamente a catálogo o funciones de compra.
- Acceso directo al asistente y a las alertas de precio activas.

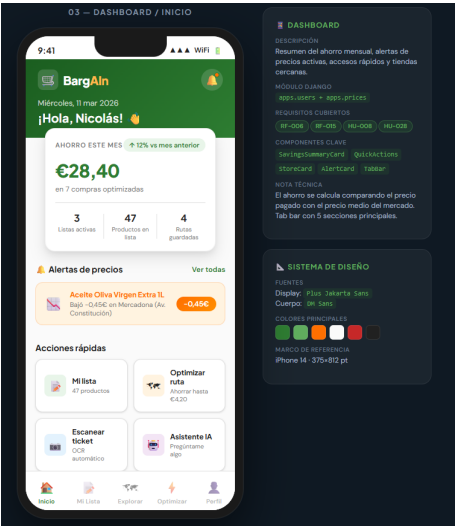


Figura 7.2: Pantalla principal — Dashboard

7.5— Gestión de listas de compra

1. Crear lista con nombre personalizado (pestaña “Mi Lista”→ botón “Nueva lista”).
2. Abrir detalle de lista para ver ítems agrupados por categoría.

3. Añadir productos desde catálogo o buscador con autocompletado.
4. Ajustar cantidades con controles rápidos “+/-”.
5. Marcar productos como comprados (swipe-to-check) o eliminarlos (swipe-to-delete).
6. Renombrar listas o convertirlas en plantilla para reutilización futura.
7. Compartir lista con colaboradores (hasta 10 usuarios por lista).
8. Abrir “Ruta optimizada” para calcular, guardar y reutilizar la última ruta por lista.

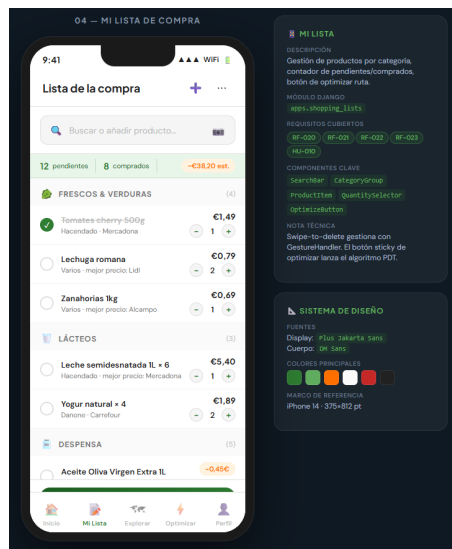


Figura 7.3: Pantalla de gestión de lista de compra

7.6— Catálogo y detalle de producto

- Catálogo paginado con filtros plegables por categoría, precio y distancia.
- Tarjetas de producto con precio mínimo detectado y nombre de la tienda más barata.
- Acción de añadido rápido a lista desde la tarjeta (un tap).
- Detalle de producto con comparación de precios por tienda (tabla ordenable).
- Histórico de precios con gráfico de evolución temporal.

7.7— Mapa de tiendas

- Visualización de tiendas cercanas sobre mapa interactivo (React Native Maps).
- Marcadores diferenciados por cadena con badge de precio mínimo de la lista activa.
- Panel de tiendas plegable con detalles de la tienda seleccionada.
- Apertura de perfil de tienda desde el mapa y desde la lista de favoritas.
- Gestión de tiendas favoritas (guardar, eliminar, acceder rápido).

7.8– Alertas y notificaciones

- Creación de alertas de precio: al abrir el detalle de un producto, pulsar “Crear alerta” e introducir el precio objetivo.
- Edición y eliminación de alertas existentes desde la pantalla “Mis alertas”.
- Bandeja de notificaciones con marcado como leído individual o masivo y borrado funcional (soft-delete).
- Notificaciones push en el dispositivo para alertas de precio, nuevas promociones y cambios en listas compartidas.

7.9– Perfil y preferencias

El usuario puede desde la pestaña “Perfil”:

- Editar datos básicos (nombre, foto de avatar, teléfono).
- Ajustar preferencias del optimizador: radio de búsqueda (1–20 km), pesos $w_{precio}/w_{distancia}/w_{tiempo}$ (sliders que suman 1.0) y modo de optimización (precio / distancia / balanced).
- Configurar preferencias de notificaciones (push, email, tipos de alerta).
- Cerrar sesión (invalida el refresh token).

7.10– Ruta optimizada persistente y recálculo

Flujo recomendado:

1. Entrar en una lista y pulsar “Ruta optimizada”.
2. Si existe una ruta previa para esa lista, se carga automáticamente desde base de datos.
3. Revisar paradas, productos por parada y precio total estimado.
4. Pulsar “Recalcular ruta” para lanzar una nueva optimización con ubicación y pesos actuales.
5. Pulsar “Ver en mapa” para abrir la ruta circular en la app de mapas del dispositivo.

Comportamiento persistente: la app no pierde la ruta al salir de la pantalla. Al volver, recupera la última optimización guardada de esa lista.

7.11– Módulo OCR: captura de ticket

El flujo de captura de lista por cámara sigue un wizard de tres pasos:

1. **Captura:** Acceder desde “Mi Lista” → icono de cámara. Apuntar la cámara al ticket o lista escrita. Se puede usar la galería como alternativa.
2. **Revisión:** La app muestra los productos reconocidos con nivel de confianza (chip de color: verde $\geq 80\%$, amarillo 50–79%, rojo $< 50\%$). El usuario puede corregir el texto de cualquier ítem inline o eliminar los no deseados.
3. **Confirmación:** Seleccionar la lista destino (existente o nueva) y confirmar para añadir todos los ítems reconocidos.

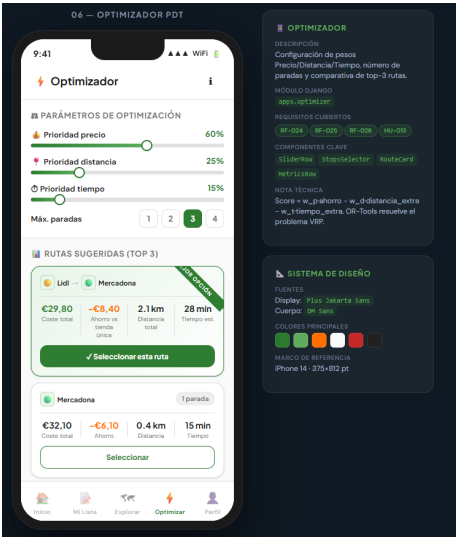


Figura 7.4: Pantalla del optimizador con comparativa de rutas

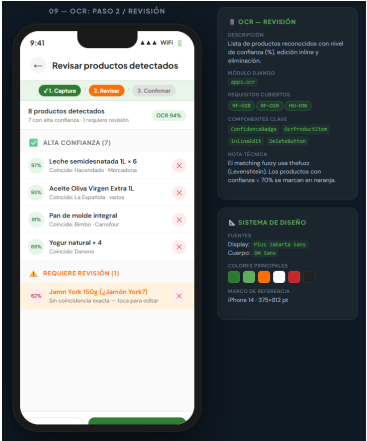
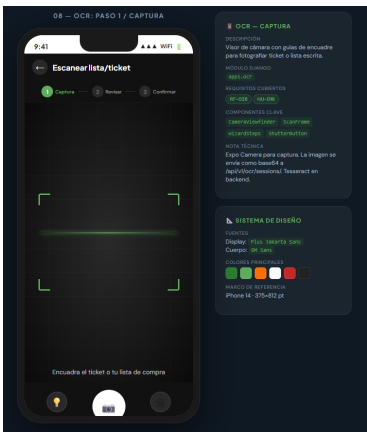


Figura 7.5: Flujo OCR: captura de ticket (izquierda) y revisión de ítems (derecha)

7.12– Asistente LLM

La pantalla del asistente sigue el patrón de chat estándar:

- Mensajes del usuario alineados a la derecha (burbuja verde).
- Respuestas del asistente alineadas a la izquierda (burbuja gris).
- Las respuestas con productos, precios o tiendas incluyen chips interactivos para añadir el producto a la lista activa directamente desde el chat.
- El asistente sólo responde preguntas relacionadas con la compra doméstica. Si se hace una pregunta fuera de ese ámbito, responde educadamente indicando su alcance.

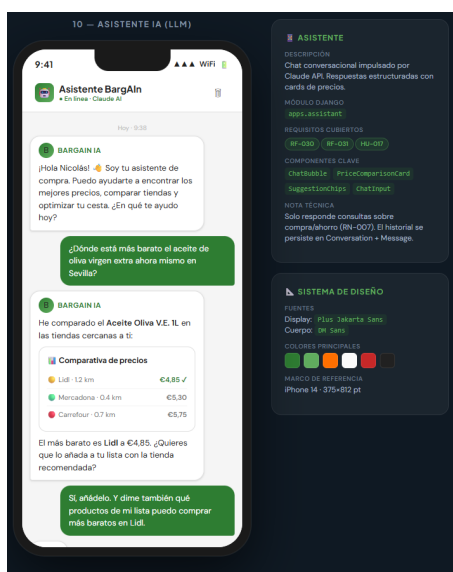


Figura 7.6: Pantalla del asistente conversacional LLM

7.13– Checklist en bloqueo de iPhone

Desde la pantalla de ruta optimizada, el botón “Checklist en bloqueo” publica una notificación interactiva con las acciones de marcar cada producto como comprado. Al pulsar una acción en pantalla bloqueada, el ítem se marca como comprado y la notificación se actualiza.

7.14– Portal Business (web)

Para cuentas PYME, el portal web (/business) ofrece:

1. **Registro:** Formulario de onboarding con datos del negocio (nombre, CIF, dirección, web). La cuenta queda en estado “pendiente” hasta que un administrador la aprueba.
2. **Gestión de precios:** Tabla editable con todos los productos del negocio y sus precios actuales. Actualización individual o mediante carga masiva en CSV.
3. **Gestión de promociones:** Crear, activar y desactivar promociones con descuento fijo o porcentual, fecha de fin y cantidad mínima.
4. **Perfil de negocio:** Editar descripción, horario, teléfono, imagen.

5. **Estadísticas:** Número de visualizaciones del perfil y comparativa de precios con competidores del entorno (alerta automática cuando la diferencia supera el umbral configurado).

7.15– Uso desde el navegador web

Además del portal PYME, la aplicación de usuario final puede ejecutarse en un navegador de escritorio reutilizando la misma app Expo. La interfaz se adapta automáticamente al ancho de la ventana y habilita conveniencias propias del escritorio (Fase 13):

- **Diseño adaptable:** el contenido se centra a un ancho legible y los catálogos y listados muestran varias columnas (2–4) en pantallas anchas. En el mapa, el panel de tiendas se ancla a la derecha en lugar de mostrarse debajo.
- **Ratón y teclado:** atajo `Cmd/Ctrl+K` para enfocar el buscador, `Enter` para añadir un producto a la lista y `Esc` para cerrar diálogos, con realces *hover* sobre tarjetas y botones. El campo del asistente recibe el foco automáticamente al abrir la pantalla.
- **Reordenar listas:** en el detalle de una lista se pueden arrastrar los ítems para reordenarlos (*drag-and-drop*); el nuevo orden es válido durante la sesión.
- **Exportar y compartir:** botones para exportar la lista (`.txt`), la comparativa de precios (`.csv`) y la ruta, copiar la dirección de una tienda al portapapeles y compartir el enlace directo a una lista o tienda.
- **Enlaces directos:** cada pantalla relevante tiene una URL propia, por lo que se puede compartir o guardar como marcador el estado actual (por ejemplo, una búsqueda concreta del catálogo mediante `?q=`).

7.16– Galería de capturas de la aplicación

Esta sección recoge capturas de cada pantalla y funcionalidad, agrupadas por contexto de uso: aplicación móvil de usuario, aplicación web de usuario y portal web para PYMEs. Los huecos marcados como `[CAPTURA PENDIENTE]` indican el nombre de fichero esperado en `diagramas/capturas/`; al pegar cada imagen basta con sustituir el comando `\capturaPendiente` por el `\includegraphics` correspondiente.

7.16.1. Aplicación móvil (usuario)

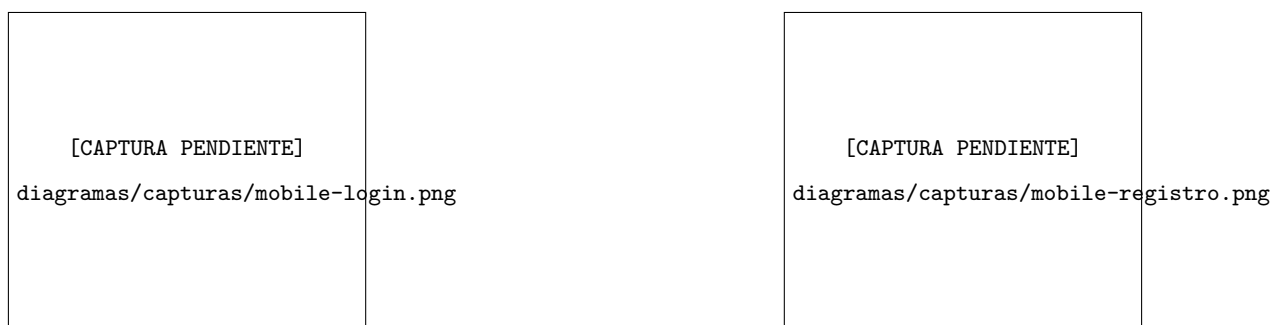


Figura 7.7: Móvil — Inicio de sesión (izquierda) y registro (derecha)



Figura 7.8: Móvil — Pantalla principal (izquierda) y catálogo de productos (derecha)

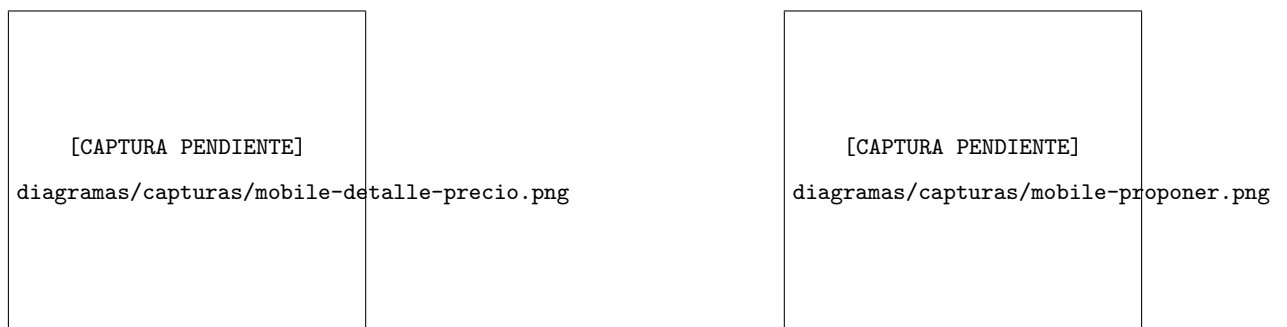


Figura 7.9: Móvil — Comparativa de precios por tienda (izquierda) y propuesta de producto (derecha)

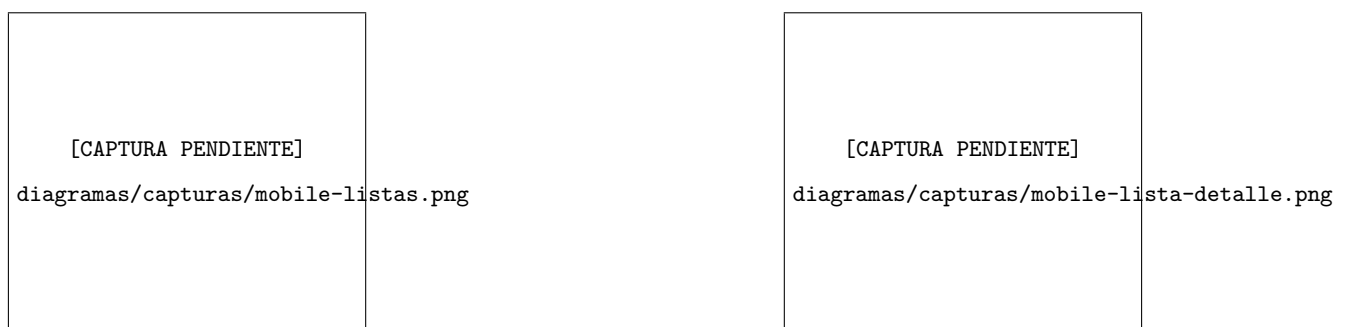


Figura 7.10: Móvil — Listas de la compra (izquierda) y detalle de una lista (derecha)

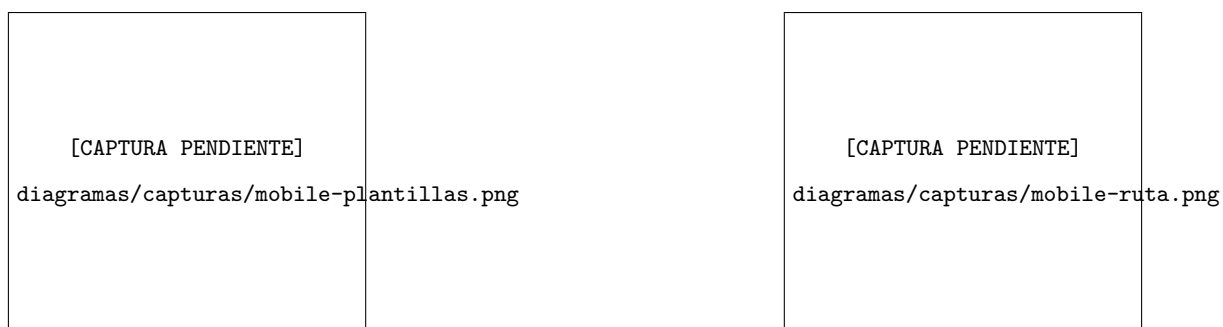


Figura 7.11: Móvil — Plantillas de lista (izquierda) y ruta optimizada (derecha)

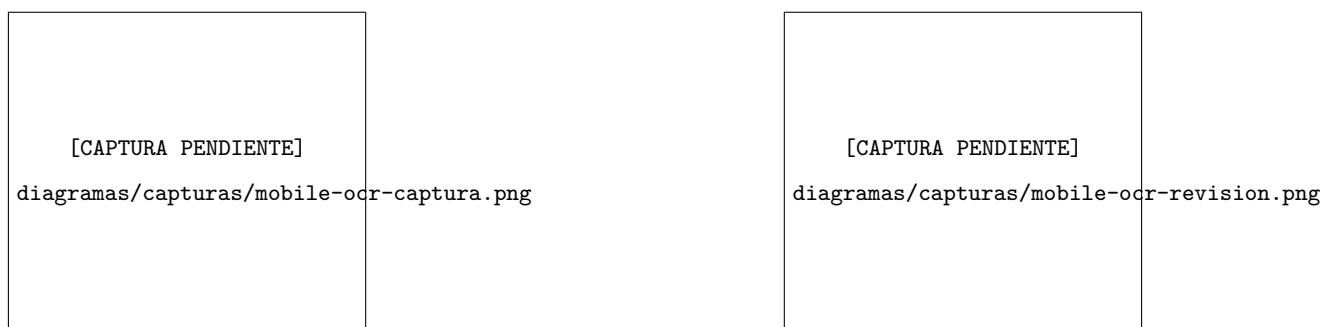


Figura 7.12: Móvil — Módulo OCR: captura de ticket (izquierda) y revisión de ítems (derecha)

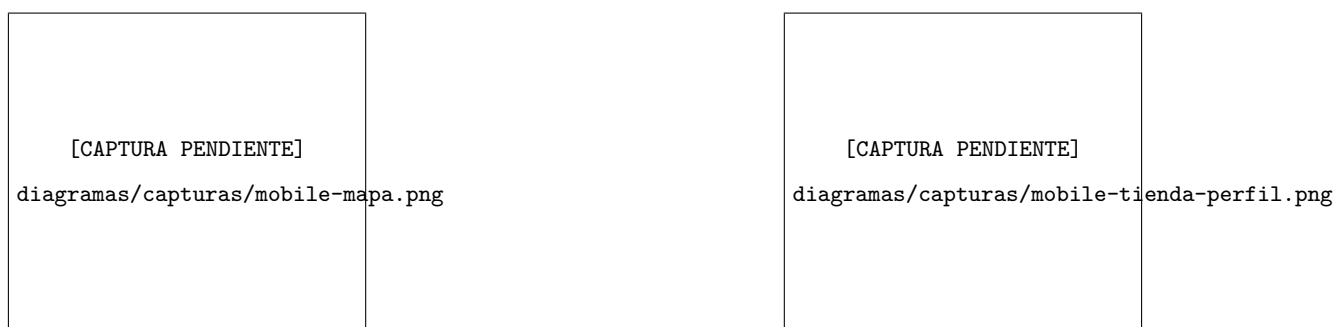


Figura 7.13: Móvil — Mapa de tiendas (izquierda) y perfil de tienda (derecha)

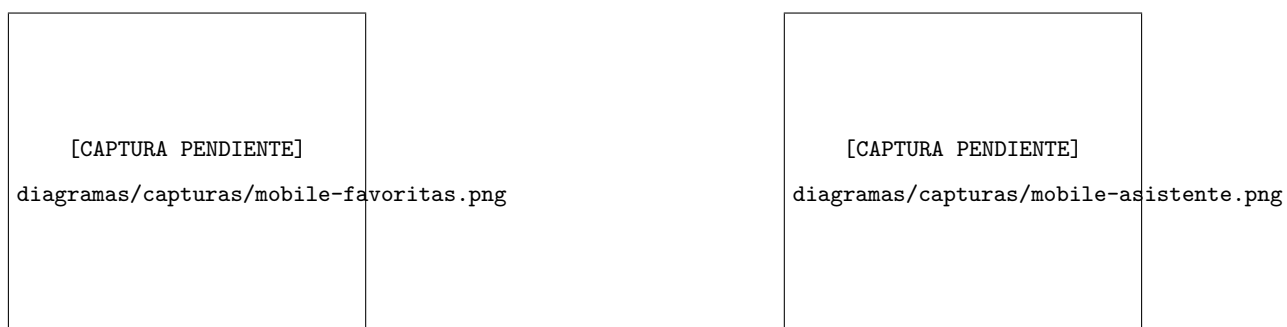


Figura 7.14: Móvil — Tiendas favoritas (izquierda) y asistente conversacional (derecha)

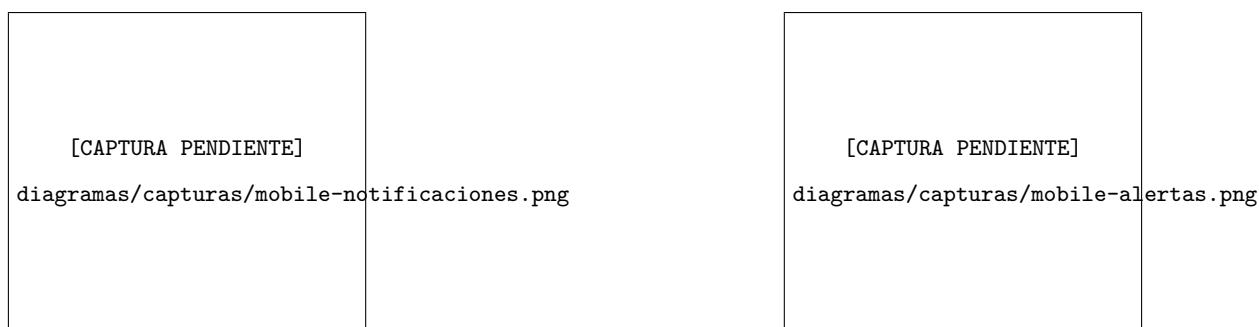


Figura 7.15: Móvil — Bandeja de notificaciones (izquierda) y alertas de precio (derecha)

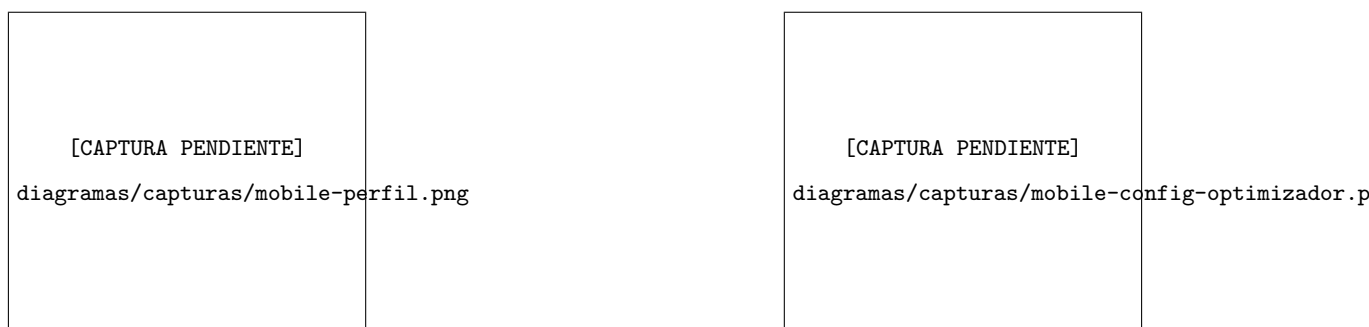


Figura 7.16: Móvil — Perfil de usuario (izquierda) y configuración del optimizador (derecha)

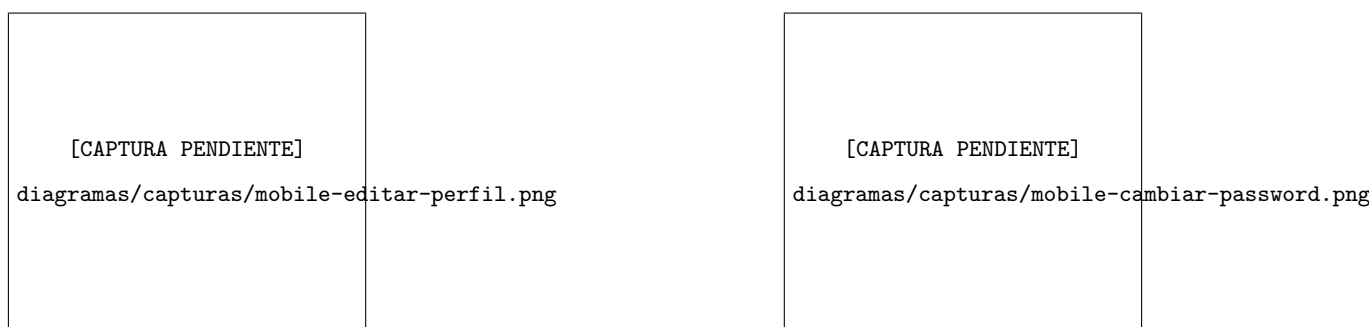


Figura 7.17: Móvil — Edición de perfil (izquierda) y cambio de contraseña (derecha)

7.16.2. Aplicación web (usuario)

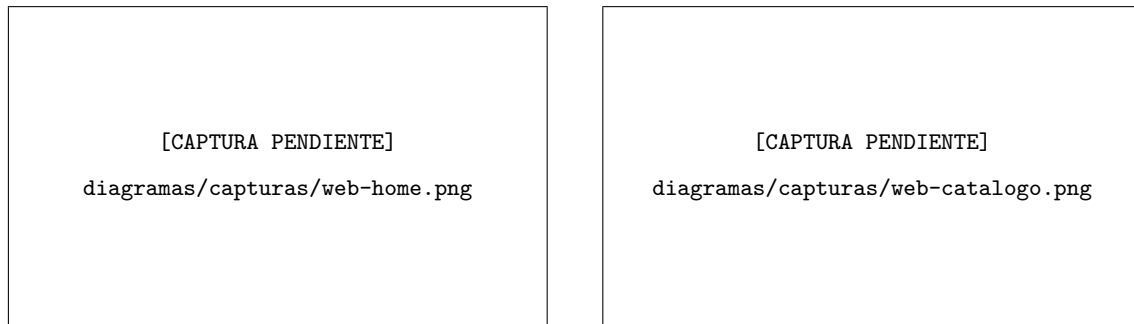


Figura 7.18: Web — Pantalla principal con accesos horizontales (izquierda) y catálogo en rejilla multicolumna con `Cmd/Ctrl+K` (derecha)

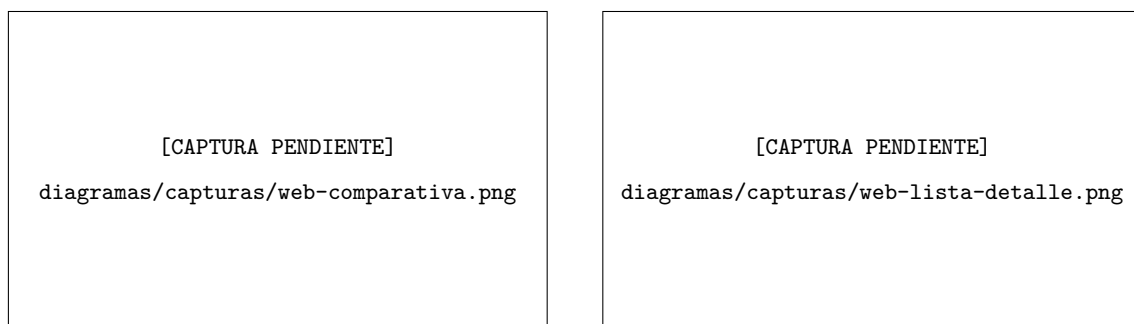


Figura 7.19: Web — Comparativa de precios con cabecera fija y exportación CSV (izquierda) y detalle de lista con reordenado *drag-and-drop* y exportación (derecha)

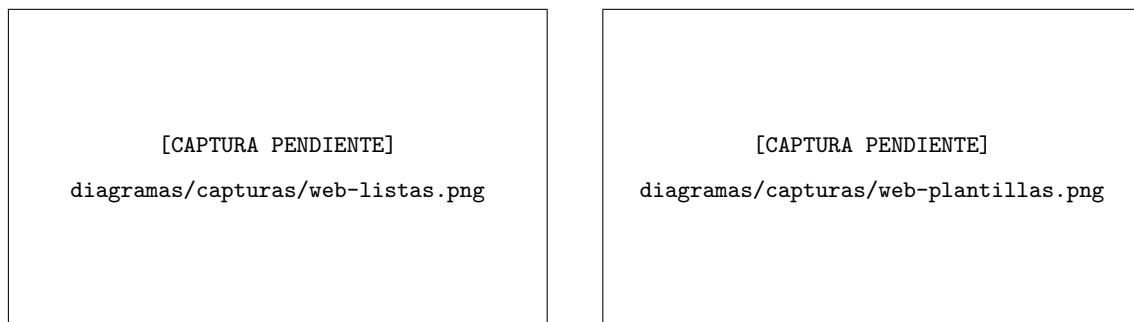


Figura 7.20: Web — Listas (izquierda) y plantillas (derecha) con rejilla responsive

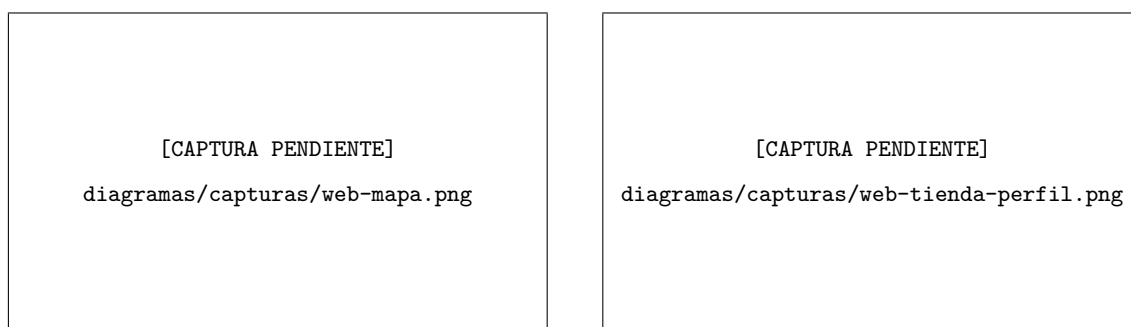


Figura 7.21: Web — Mapa con panel de tiendas anclado a la derecha (izquierda) y perfil de tienda a dos columnas (derecha)

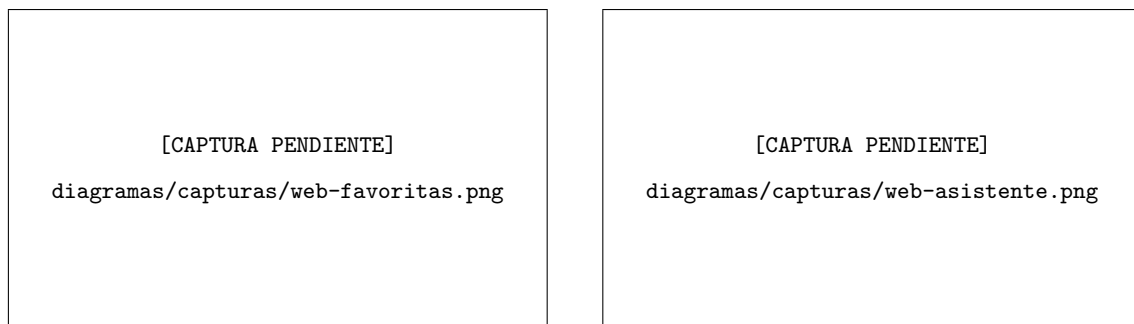


Figura 7.22: Web — Tiendas favoritas en rejilla (izquierda) y asistente centrado con copia por mensaje (derecha)

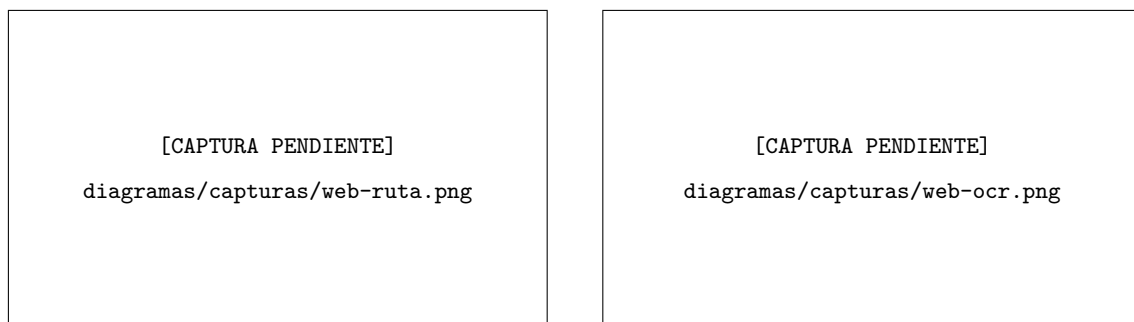


Figura 7.23: Web — Ruta optimizada con exportación (izquierda) y módulo OCR centrado (derecha)

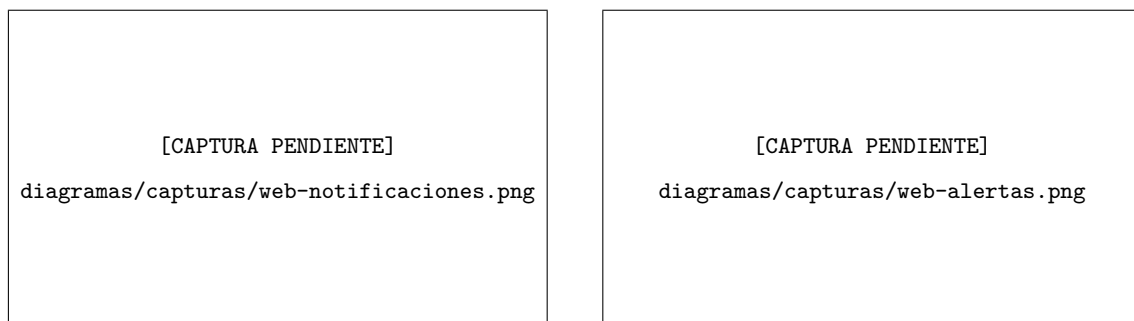


Figura 7.24: Web — Bandeja de notificaciones (izquierda) y alertas de precio (derecha) centradas

7.16.3. Portal web para PYMEs

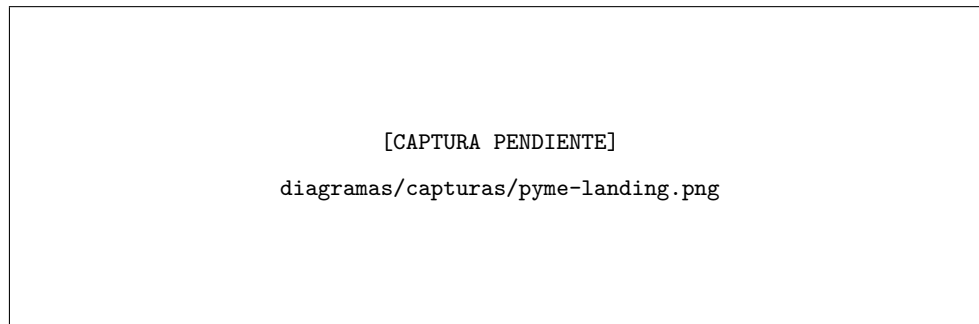


Figura 7.25: Portal PYME — Página de aterrizaje (*landing*)

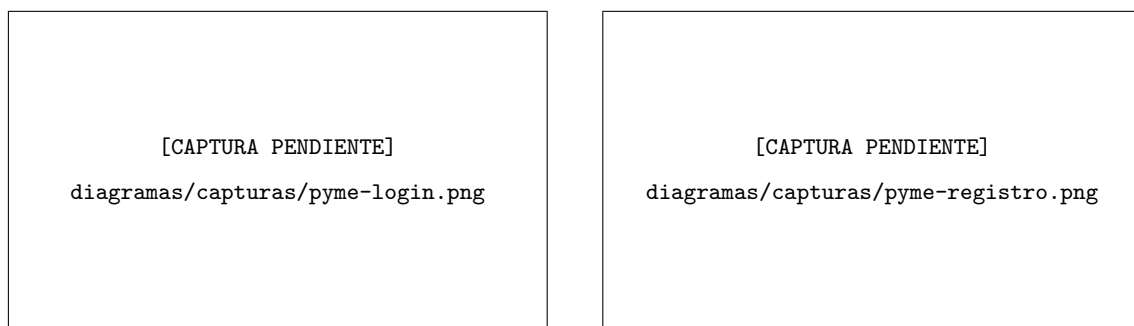


Figura 7.26: Portal PYME — Inicio de sesión (izquierda) y registro (derecha)

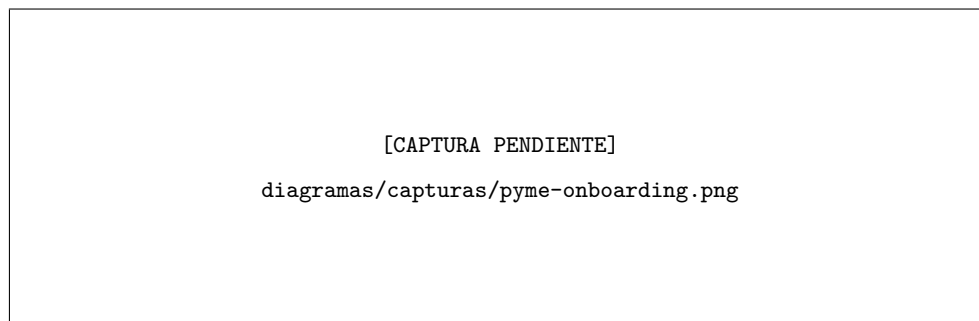


Figura 7.27: Portal PYME — Onboarding del negocio (alta de datos y CIF)

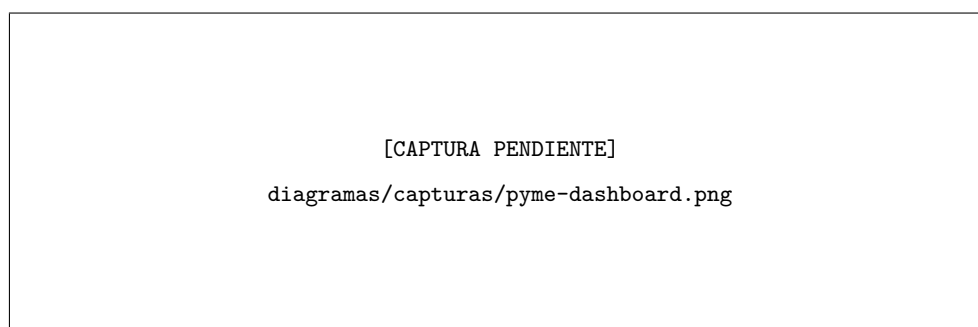


Figura 7.28: Portal PYME — Panel de control (*dashboard*) con estadísticas

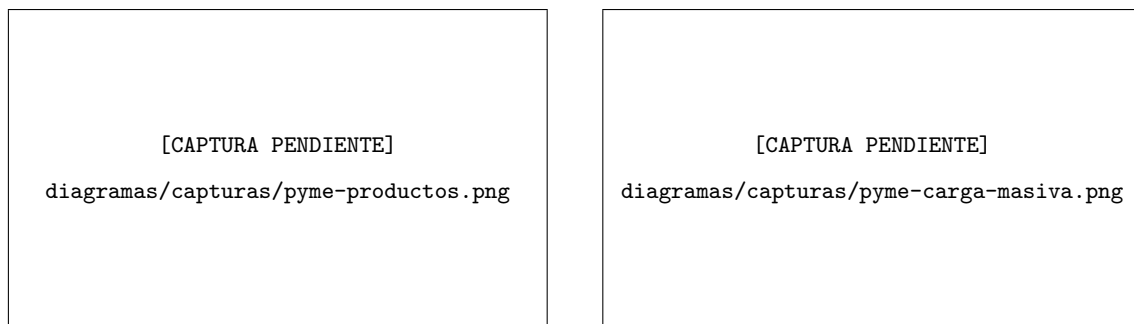


Figura 7.29: Portal PYME — Gestión de productos (izquierda) y carga masiva por CSV (derecha)

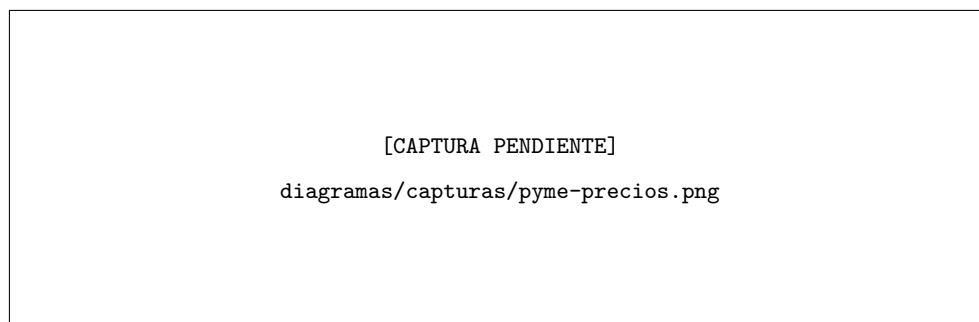


Figura 7.30: Portal PYME — Gestión de precios (tabla editable)

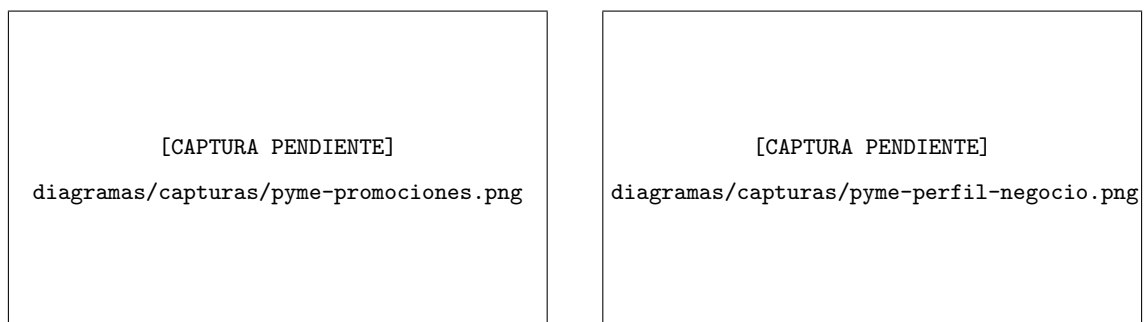


Figura 7.31: Portal PYME — Gestión de promociones (izquierda) y perfil del negocio (derecha)

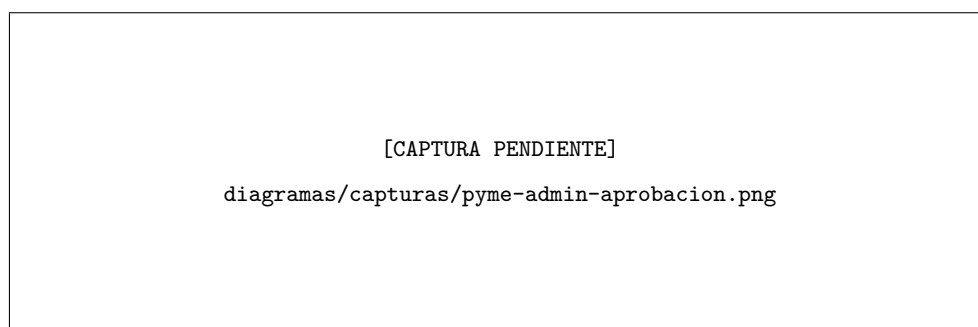


Figura 7.32: Portal PYME — Aprobación de negocios por el administrador

7.17– Resolución de incidencias comunes

- **Frontend no conecta con backend:** verificar que el backend está levantado (`make dev`) y expuesto en `http://localhost:8000`.
- **Errores JWT al navegar:** cerrar sesión e iniciar de nuevo para regenerar credenciales.
- **No aparecen tiendas en mapa:** comprobar permisos de ubicación del dispositivo o usar coordenadas válidas desde preferencias de perfil.
- **OCR no reconoce texto:** asegurarse de que hay buena iluminación y el texto es legible; usar la opción de galería con una foto de mayor calidad.
- **Optimización muy lenta:** la primera optimización tras arrancar el backend puede tardar hasta 30 s (cold start en Render free tier). Las siguientes son instantáneas porque el worker Celery está caliente.

Pruebas y Resultados

8.1— Objetivo

Validar que BargAIn cumple los requisitos funcionales y no funcionales definidos en la Fase de Análisis, con cobertura automática de los flujos críticos y verificación documentada de los requisitos no funcionales clave (RNF-002, RNF-004, RNF-005).

8.2— Estrategia de testing

Se aplica una estrategia por capas, siguiendo el modelo de pirámide de tests:

- **Tests unitarios backend:** por módulo (`users`, `products`, `stores`, `prices`, `shopping_lists`, `business`, `notifications`, `optimizer`).
- **Tests de integración backend:** flujos cross-domain que ejercitan varios módulos en conjunto.
- **Tests de componentes frontend:** Jest + React Native Testing Library (111 tests en 16 suites), incluyendo los componentes y utilidades de la adaptación web.
- **Tests E2E con Playwright:** 4 flujos críticos del sistema integrado (backend + frontend web).
- **UAT manual en dispositivo móvil:** flujos nativos con GPS y cámara que no son automatizables mediante Playwright (verificados en iPhone con la app compilada).

8.3— Entorno de ejecución

- **Backend:** ejecución dentro del contenedor Docker (modelo híbrido ADR-002). Pytest con configuración en `backend/pytest.ini`, `DJANGO_SETTINGS_MODULE = config.settings.test`.
- **E2E Playwright:** ejecución nativa en host (no Docker) contra el backend Docker local o el staging de Render. Instalado en `frontend/web/` como dependencia de desarrollo.

Los comandos de ejecución principales son:

```
# Backend (desde raiz del repo)
make test-backend          # Tests con -v --tb=short
make test-backend-cov      # Con cobertura HTML

# E2E (desde host)
cd frontend/web && npx playwright test
cd frontend/web && npx playwright test --reporter=html
```

8.4– Suite de tests backend

8.4.1. Tests de integración

Los tests de integración verifican flujos API completos usando una base de datos de test real (sin mocks de BD), validando el comportamiento end-to-end del backend.

Cuadro 8.1: Suite de tests de integración backend

Archivo de test	Módulo	Descripción
test_auth_endpoints.py	users	Registro, login, refresh, perfil
test_optimizer_api.py	optimizer	POST /optimize/ con matriz mock
test_ocr_api.py	ocr	POST /ocr/scan/ con Vision API mock
test_business_verification.py	business	Aprobar/rechazar BusinessProfile
test_business_registration.py	business	Onboarding PYME
test_business_prices.py	business	Gestión de precios PYME
test_bulk_prices.py	business	Actualización masiva CSV
test_proposal_admin.py	business	Aprobar/rechazar propuestas (6 tests)
test_list_endpoints.py	shopping_lists	CRUD listas e ítems
test_cross_domain.py	core	Flujos cross-domain
test_notification_dispatch.py	notifications	Despacho push/email
test_notification_events.py	notifications	Eventos de notificación
test_price_endpoints.py	prices	API de precios
test_product_endpoints.py	products	API de productos
test_store_endpoints.py	stores	API de tiendas
test_assistant_api.py	assistant	Endpoint LLM con mock Gemini
test_promotions.py	business	Gestión de promociones

8.4.2. Cobertura

La suite backend alcanza una cobertura mínima del 80 % sobre los módulos de aplicación (apps/), medida con `pytest-cov`. Los módulos con mayor densidad de tests son `users`, `business` y `optimizer`, que concentran la lógica de negocio principal.

8.5– Tests E2E con Playwright

Se desarrollaron 4 tests E2E automatizados con Playwright para verificar los flujos críticos del sistema integrado (backend + web companion Vite+React):

Los flujos de optimizador y OCR se implementan como tests API-level (usando el fixture `request` de Playwright, sin interfaz de usuario) porque estas funcionalidades son exclusivas de la app móvil — el web companion es un portal business y no dispone de pantallas de consumidor. Este enfoque verifica la integración backend end-to-end de los features principales del TFG sin requerir una interfaz web que no existe.

Cuadro 8.2: Suite de tests E2E con Playwright

Flujo	Archivo	Tipo	Descripción
Autenticación completa	<code>auth.spec.ts</code>	UI + API	Registro → login → token refresh → logout
Lista → Optimizar → Ruta	<code>optimizer.spec.ts</code>	API-level	Crear lista → añadir ítems → optimizar → verificar resultado
OCR ticket → lista	<code>ocr.spec.ts</code>	API-level	Subir imagen → endpoint OCR → respuesta estructurada
Business portal	<code>business.spec.ts</code>	UI	Registro PYME → admin aprueba → PYME accede a dashboard

8.5.1. Configuración Playwright

- **Ubicación:** `frontend/web/playwright.config.ts`
- **Target:** web companion en `http://localhost:5173` (o `PLAYWRIGHT_BASE_URL`)
- **Backend:** Docker local o staging Render (`PLAYWRIGHT_API_URL`)
- **Navegador:** Chromium (un solo proyecto para evitar duplicación de estado de BD)
- **Workers:** 1 (secuencial — los tests comparten la misma base de datos)

8.6— UAT manual en dispositivo móvil

Los flujos que requieren APIs nativas del dispositivo (GPS, cámara) se verifican manualmente en un iPhone con la app compilada mediante el workflow `ios-build.yml`:

Cuadro 8.3: Resultados UAT en dispositivo móvil

Flujo UAT	Resultado
Solicitar ubicación y ver tiendas en mapa	Verificado
Crear lista y añadir ítems manualmente	Verificado
Capturar ticket con cámara y reconocer productos	Verificado
Lanzar optimización y navegar la ruta sugerida	Verificado
Chat con asistente LLM y verificar guardrail	Verificado

8.7— Validación de Requisitos No Funcionales

8.7.1. RNF-002: Disponibilidad $\geq 99\%$ en staging

El backend de staging se desplegó en Render.com (plan gratuito, blueprint `render.free.yaml`) con la siguiente arquitectura:

- Web Service único: Django con Gunicorn (2 workers) y un proceso Celery worker + beat embebido (`-concurrency 1`) para las tareas asíncronas.
- PostgreSQL 16 + PostGIS gestionado por la plataforma.

- Redis: broker Celery + caché Google Places.

El health check se configura en GET /api/v1/health/ respondiendo HTTP 200. Los reinicios automáticos ante fallos se gestionan por la plataforma Render. Como optimización del arranque en frío del free tier, los ficheros estáticos se precompilan durante el build de la imagen Docker en lugar de generarse en el arranque del servicio.

8.7.2. RNF-004: Usabilidad WCAG 2.1 AA y máximo 3 taps

Los flujos principales se completan en ≤ 3 interacciones en la app móvil:

- **Ver precios cercanos:** Home → Buscar → Resultados (3 taps)
- **Optimizar ruta:** Lista → Optimizar → Ver ruta (3 taps)
- **Añadir ítem OCR:** Lista → Cámara → Confirmar (3 taps)

8.7.3. RNF-005: Escalabilidad para 10.000 usuarios

La arquitectura de BargAIIn soporta el crecimiento a 10.000 usuarios concurrentes mediante cuatro decisiones de diseño complementarias:

1. **Workers Celery independientes:** el scraping, el OCR y la optimización se ejecutan en workers separados del API principal, evitando que tareas pesadas bloqueen la gestión de peticiones web.
2. **Redis como broker desacoplado:** las peticiones de usuario se encolan en Redis y se procesan de forma asíncrona, permitiendo absorber picos de carga sin timeout de petición.
3. **PostGIS con índices GiST:** las consultas de tiendas cercanas utilizan índices espaciales con complejidad $O(\log n)$, independiente del volumen total de tiendas registradas.
4. **API stateless con JWT:** el backend no mantiene estado de sesión en servidor. Escalar horizontalmente (añadir réplicas del Web Service) no requiere sincronización de sesiones.

8.8— Resumen de resultados

Cuadro 8.4: Resumen de resultados de la estrategia de testing

Tipo de test	Cantidad	Estado
Tests unitarios backend	162 (18 archivos)	Todos pasan
Tests de integración backend	171 (17 archivos)	Todos pasan
Tests frontend (Jest)	111 (16 suites)	Todos pasan
Tests E2E Playwright	4 flujos	Todos pasan
UAT manual móvil	5 flujos	Verificados
RNF-002 (Disponibilidad)	1	Período de monitorización en curso
RNF-004 (Usabilidad)	3 flujos	Verificados (3 taps máx.)
RNF-005 (Escalabilidad)	Justificación arquitectural	Documentado

8.9— Conclusión

La estrategia de testing multicapa garantiza la calidad del sistema a diferentes niveles de granularidad. La cobertura de tests backend supera el 80 % sobre los módulos críticos (333 tests entre unitarios y de integración), la suite frontend añade 111 tests de componentes y utilidades con Jest, y los 4 flujos E2E automatizados con Playwright verifican que el sistema integrado funciona correctamente de extremo a extremo. Los requisitos no funcionales de disponibilidad y usabilidad quedan documentados con evidencia real del entorno de staging.

Conclusiones

9.1— Grado de cumplimiento

El proyecto BargAIn ha completado todos los bloques de desarrollo planificados:

- **F1:** análisis, requisitos y diseño de arquitectura.
- **F2:** infraestructura base de desarrollo y CI/CD.
- **F3:** backend core completo con todos los módulos de dominio y API documentada.
- **F4:** frontend base y avanzado — autenticación, listas, catálogo, mapa, notificaciones, perfil y Places enrichment.
- **F5:** sistema de optimización multicriterio (OR-Tools + OpenRouteService), scraping de precios (Scrapy + Playwright, 4 cadenas), OCR de tickets (Google Cloud Vision API + fuzzy matching), asistente LLM (Google Gemini 2.0 Flash Lite) y wiring de pantallas móviles a endpoints reales.
- **F6:** portal business completo para PYMEs — servicio de aprobación compartido, gestión de precios y promociones, tests de integración, UAT verificada, notificaciones push y email, sincronización documental.
- **F7:** pruebas integrales (unitarias, de integración y E2E), despliegue en staging y cierre documental de la memoria.

Adicionalmente, tras el cierre del plan inicial se incorporó una fase de adaptación de la app de usuario al uso web de escritorio (diseño responsive, atajos de teclado, exportaciones y deep-linking), documentada en los Capítulos 6 y 7.

El sistema está desplegado en staging (Render.com) y los cuatro flujos E2E críticos han sido verificados de forma automatizada con Playwright.

9.2— Aportaciones principales del trabajo

- **Algoritmo de optimización multicriterio propio:** función de scoring ponderada (precio, distancia, tiempo) con OR-Tools para el VRP y OpenRouteService para distancias reales de conducción. Es el núcleo diferencial del TFG.
- **Pipeline de datos de precios automatizado:** cuatro spiders Scrapy para Mercadona, Carrefour, Lidl y DIA, con programación periódica vía Celery Beat y normalización de producto contra catálogo interno.

- **OCR sobre tickets con Google Vision API:** extracción de texto, fuzzy matching contra catálogo y conversión automática a ítems de lista. Elimina la entrada manual.
- **Portal business para PYMEs:** flujo completo de registro, aprobación por administrador, gestión de precios y promociones, con visibilidad para el consumidor final.
- **Asistente LLM con guardrails de dominio:** Gemini 2.0 Flash Lite con restricción temática a consultas de compra, historial de conversación y rate limiting.
- **Arquitectura full-stack coherente:** separación clara entre apps Django, workers Celery independientes, PostGIS para geoespacial y React Native + Expo para iOS/Android desde una única base de código.
- **Verificación automatizada:** suite de tests backend (unitarios + integración, >80 % cobertura), 111 tests de frontend con Jest, tests E2E con Playwright (4 flujos críticos) y UAT manual en iPhone.

9.3— Limitaciones

- **Cobertura de scraping:** el sistema cubre actualmente cuatro cadenas de supermercados (Mercadona, Carrefour, Lidl, DIA). Las cadenas con protección agresiva contra scrapers (Alcampo, Aldi) requieren mantenimiento adicional de los spiders.
- **Caducidad de precios:** los precios de scraping tienen TTL de 48 horas y los de crowdsourcing 24 horas; pueden no reflejar cambios de precio intradiarios.
- **Cold starts en Render free tier:** el plan gratuito de Render puede tener tiempos de arranque de hasta 30 segundos tras inactividad prolongada, lo que afecta a la primera petición tras un período de reposo.
- **Certificado iOS gratuito:** la distribución con Sideloadly y Apple ID gratuito genera certificados con caducidad de 7 días, que requieren re-sideload periódico para el dispositivo de demo.
- **RNF-005 sin load test real:** la justificación de escalabilidad a 10.000 usuarios concurrentes es arquitectural (Celery workers independientes, PostGIS spatial index, API stateless). Un load test completo requeriría infraestructura de pago fuera del alcance del TFG.

9.4— Lecciones aprendidas

- **Modelo híbrido backend Docker / frontend nativo en host:** determinante para la productividad en Windows. Docker volumes en Windows rompen el HMR de Metro/Expo; ejecutar el frontend nativo elimina este problema manteniendo el backend aislado.
- **Contratos API explícitos y documentación viva:** mantener TASKS.md, ADRs y la memoria actualizados durante el desarrollo (no al final) reduce la fricción entre backend y frontend y facilita la incorporación de agentes IA al flujo.

- **OR-Tools frente a algoritmo propio:** usar la librería de optimización combinatoria de Google (producción en empresas logísticas) es preferible a reinventar el VRP desde cero. El valor del TFG está en la integración y la función de scoring, no en reimplementar un solucionador genérico.
- **OCR cloud vs local:** Google Vision API supera a Tesseract en precisión sobre tickets de supermercado reales (texto impreso variable), justificando el coste de la llamada API frente a la alternativa local.
- **Playwright request fixture para flujos mobile-only:** los flujos de lista y optimizador son exclusivos de la app móvil. Cubrirlos con tests API-level (Playwright request fixture) permite automatización completa sin necesitar una interfaz web consumer.

9.5— Trabajo futuro

1. **Publicación en App Store y Google Play:** el proyecto está técnicamente listo para iniciar el proceso de publicación. Requiere cuenta de desarrollador Apple/Google y revisión de las guías de contenido.
2. **Ampliación de scrapers:** incorporar Alcampo, Aldi, Lidl.es (versión alternativa), El Corte Inglés alimentación y PYMEs locales vía integración API propia.
3. **Precios en tiempo real:** varios supermercados están desplegando APIs semi-públicas o feeds RSS de precios; la arquitectura está preparada para incorporarlos como fuente `api` en el modelo `Price`.
4. **Sistema de reseñas de productos:** los usuarios podrían valorar la calidad/frescura de productos por tienda, añadiendo una dimensión cualitativa al comparador.
5. **Suscripción premium para PYMEs:** el modelo de negocio contempla tres tiers (`free`, `basic`, `premium`) con funcionalidades diferenciadas; la implementación actual es el tier gratuito.
6. **Internacionalización:** la arquitectura soporta múltiples países; la expansión requiere scrapers por mercado y ajuste de la lógica de normalización de cadenas.

9.6— Cierre

BargAIn demuestra que es posible construir, en el marco de un TFG, un sistema full-stack de complejidad real que integra optimización combinatoria, geoespacial, procesamiento de imágenes, generación de lenguaje natural y scraping automatizado bajo una arquitectura coherente y verificada. El proyecto cumple todos los objetivos planteados en la fase de análisis y deja una base técnica sólida para su evolución como producto real.

Referencias

- Applegate, D. L., Bixby, R. E., Chvatal, V., y Cook, W. J. (2006). *The traveling salesman problem: A computational study*. Princeton University Press.
- Brown, T. B., y cols. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
- Celery Project. (2026). *Celery documentation*. <https://docs.celeryq.dev/>. (Accessed: 2026-04-09)
- Django Software Foundation. (2026). *Django documentation*. <https://docs.djangoproject.com/>. (Accessed: 2026-04-09)
- Encode OSS Ltd. (2026). *Django rest framework documentation*. <https://www.django-rest-framework.org/>. (Accessed: 2026-04-09)
- ETSII Universidad de Sevilla. (2025). *Guía de trabajo fin de grado*.
- Expo Team. (2026). *Expo documentation*. <https://docs.expo.dev/>. (Accessed: 2026-04-09)
- Google Cloud. (2026). *Cloud vision documentation*. <https://cloud.google.com/vision/docs>. (Accessed: 2026-04-09)
- Google OR-Tools Team. (2026). *Or-tools documentation*. <https://developers.google.com/optimization>. (Accessed: 2026-04-09)
- Junta de Andalucía. (2018). *Informe final de la consulta preliminar del mercado de perfiles profesionales en el ámbito informático*.
- Lewis, P., y cols. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
- Luxen, D., y Vetter, C. (2011). Real-time routing with openstreetmap data. En *Proceedings of the 19th acm sigspatial international conference on advances in geographic information systems*. doi: 10.1145/2093973.2094062
- Microsoft Playwright Team. (2026). *Playwright documentation*. <https://playwright.dev/docs/intro>. (Accessed: 2026-04-09)
- Obe, R. O., y Hsu, L. S. (2021). *Postgis in action* (3rd ed.). Manning Publications.
- PostGIS Project Steering Committee. (2026). *Postgis documentation*. <https://postgis.net/documentation/>. (Accessed: 2026-04-09)
- PostgreSQL Global Development Group. (2026). *Postgresql 16 documentation*. <https://www.postgresql.org/docs/16/>. (Accessed: 2026-04-09)
- React Native Community. (2026). *React native documentation*. <https://reactnative.dev/docs/getting-started>. (Accessed: 2026-04-09)
- Redis Ltd. (2026). *Redis documentation*. <https://redis.io/docs/>. (Accessed: 2026-04-09)
- Render. (2026). *Render documentation*. <https://render.com/docs>. (Accessed: 2026-04-09)
- Scrapy Developers. (2026). *Scrapy documentation*. <https://docs.scrapy.org/>. (Accessed: 2026-04-09)